Artificial Intelligence & Machine Learning

Unit III: Knowledge Representation, Logic and AI Languages

Study Notes with Mind Maps, Flowcharts, Diagrams and Cheat Sheet

B.C.A. Full Stack · School of Computer & Systems Sciences · Jaipur National University, Jaipur · Course code BCAFSSE501T23 (3 credits, 100 marks)

Contents

1. Unit 3 at a Glance
2. Knowledge Representation
3. Inference and Logical Reasoning
4. Propositional Logic
5. Predicate Logic (First-Order Logic)
6. Forward Chaining and Backward Chaining
7. AI Languages and Tools: Lisp, Prolog and CLIPS
8. Unit 3 Cheat Sheet
9. Important Questions
10. MCQ Practice
11. Glossary

Unit 3 at a Glance

Unit 3 is about how an intelligent system **stores what it knows** and how it **uses that knowledge to reach new conclusions**. A search algorithm (Unit 2) can find a path to a goal, but it cannot answer questions such as "Is Socrates mortal?" or "Which animal has black stripes and eats meat?" For that, an agent needs a **knowledge base**, a precise **language** to write the knowledge in (logic), and an **inference procedure** that derives new facts from old ones. The unit ends with three classic programming tools built around these ideas: **Lisp**, **Prolog** and **CLIPS**.

Syllabus Coverage

Every phrase of the official syllabus is mapped below to the chapter of these notes in which it is taught.

Table 3.1: Syllabus phrase to chapter mapping

Syllabus phrase	Where it is covered
Representation	Knowledge Representation (types, properties, approaches, KB agent)
Inference	Inference and Logical Reasoning (deduction, induction, abduction, soundness)
Propositional Logic	Propositional Logic (syntax, truth tables, laws, CNF, resolution)
Predicate logic (first order logic)	Predicate Logic (quantifiers, English to FOL, unification, FOL resolution)
Logical reasoning	Inference and Logical Reasoning (types of reasoning, monotonic vs non-monotonic)
Forward chaining	Forward and Backward Chaining (algorithm, worked examples, proof tree)
Backward chaining	Forward and Backward Chaining (algorithm, worked examples, comparison)
AI languages and tools	AI Languages and Tools (overview, Python for AI)
Lisp	AI Languages and Tools: Lisp section with tested code
Prolog	AI Languages and Tools: Prolog section with family-tree program
CLIPS	AI Languages and Tools: CLIPS section with rules and run output

Learning Outcomes

After studying this unit you should be able to:

- Define knowledge representation and list the types of knowledge and the properties of a good representation system.
- Draw and explain a semantic network and a frame, and write production rules.
- Describe a knowledge-based agent using the TELL and ASK operations.
- Distinguish deduction, induction and abduction, and define soundness and completeness.
- Write the truth table of every propositional connective and classify a sentence as a tautology, contradiction or contingency.

- Apply equivalence laws, convert any propositional sentence to Conjunctive Normal Form (CNF) and prove a goal by resolution refutation.
- Translate English sentences into first-order logic using ∀ and ∃ correctly.
- Find the Most General Unifier (MGU) of two expressions and carry out a full FOL resolution proof.
- Trace forward chaining and backward chaining on a rule base, draw the proof tree, and compare the two methods.
- Read and write simple programs in Lisp, Prolog and CLIPS, and compare the three tools.

How to Use These Notes

- **First reading:** go chapter by chapter. Each chapter starts with the idea, then gives definitions, a diagram, tables and worked examples.
- **Callout boxes:** "Key Point" boxes hold the one idea you must not miss; "Exam Tip" boxes tell you how questions are usually asked; "Memory Trick" boxes give mnemonics; "Worked Example" boxes show complete solutions.
- **Worked examples:** cover the answer, solve the problem yourself, then compare. Resolution, unification and chaining traces are favourite 10-mark questions.
- **Code:** every Lisp, Prolog and CLIPS snippet in these notes has been run, and the outputs shown are the real outputs. Typing them into an interpreter is the fastest way to learn.
- **Revision:** use the Cheat Sheet the night before the exam, then attempt the Important Questions and the MCQ Practice.

> **Key Point:** The whole unit rests on one equation: Knowledge Base + Inference Engine = Reasoning System. Representation decides what the knowledge base looks like; logic gives it precise meaning; inference (resolution, forward chaining, backward chaining) does the reasoning; Lisp, Prolog and CLIPS are tools that implement it.

Unit Mind Map



Figure 3.1: Unit 3 mind map

Table 3.2: How this unit is usually examined

Mark weight	Typical question in this unit
2 marks	Define tautology; state modus ponens; what is an MGU; what is CAR in Lisp
5 marks	Explain semantic networks / frames; compare forward and backward chaining
10 marks	Convert to CNF and prove by resolution; FOL resolution proof; Prolog program

Exam Tip: In almost every paper there is one long question that gives a short English story ("Marcus was a man…", "It is a crime for an American to sell weapons…") and asks you to (a) translate it to FOL, (b) convert to clauses and (c) prove a conclusion by resolution or chaining. Practise this pattern until you can do it in 20 minutes.

Knowledge Representation

What is Knowledge Representation?

Humans solve problems using what they know: that fire is hot, that a train leaves at 9 a.m., that a cat is a mammal. An AI system can only behave intelligently if this kind of knowledge is stored inside it in a form that a computer can **process**.

Knowledge Representation (KR) is the part of AI that deals with how knowledge about the world is written down in a formal, machine-usable form, so that a program can use it to reason, answer questions and take decisions.

A KR scheme has two sides:

- **Syntax** — the symbols and rules for writing correct sentences (how the knowledge looks).
- **Semantics** — the meaning of those sentences, i.e. what each sentence says about the world.

A related term is **Knowledge Representation and Reasoning (KRR)**, which stresses that representation is never an end in itself: it is chosen so that useful **reasoning** becomes possible.

Data, Information and Knowledge

Table 3.3: From data to knowledge

Level	Meaning	Example
Data	Raw symbols with no context	39.5
Information	Data with context	Ravi's temperature is 39.5 °C
Knowledge	Information plus rules for using it	If temperature is above 38 °C, the person has fever
Wisdom	Knowing when and why to apply knowledge	Send Ravi to a doctor, not just give medicine

Why is Knowledge Representation Needed?

- **To answer questions** that are not stored directly. If the system knows "All mammals have fur" and "A cat is a mammal", it can answer "Does a cat have fur?" without storing that fact.
- **To reason and infer** new facts from old ones.
- **To learn** — new experience must be added to existing knowledge in a consistent way.
- **To explain** decisions. An expert system can show the rules it used.
- **To communicate** — knowledge written in a standard form can be shared among programs and people.
- **To solve complex problems** such as medical diagnosis, planning, natural language understanding and robotics.

> **Key Point:** Knowledge representation is not just storage. A database also stores facts, but a KR system stores facts and rules in a form from which new, unstored facts can be derived by inference.

What Knowledge Must be Represented?

An AI system usually needs to represent the following kinds of things:

- **Objects** — things in the world: cars, students, the Taj Mahal.

- **Events** — actions that happen: "Ravi passed the exam in 2024".
- **Performance** — how to do things (behaviour, skills).
- **Meta-knowledge** — knowledge about what we know.
- **Facts** — truths about the world: "Delhi is the capital of India".
- **Knowledge base** — the collection of all the above, written as sentences.

Types of Knowledge

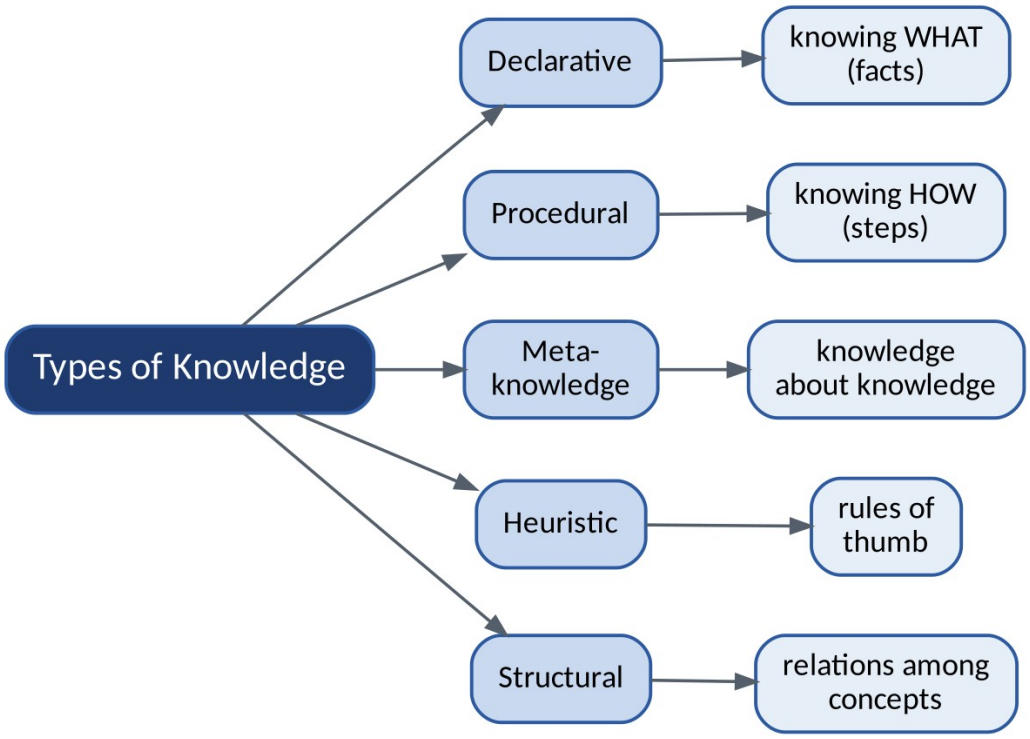


Figure 3.2: Types of knowledge

Declarative Knowledge

Declarative knowledge is knowledge *about* something — facts, concepts and objects. It is "knowing what". It is expressed in declarative sentences and is simpler than procedural knowledge.

Example: "Jaipur is the capital of Rajasthan." "Water boils at 100 °C at sea level."

Procedural Knowledge

Procedural knowledge (also called imperative knowledge) is "knowing how" to do something. It consists of rules, strategies, procedures and agendas, and can be applied directly to a task.

Example: the steps for solving a quadratic equation, or the procedure for changing a car tyre.

Meta-Knowledge

Meta-knowledge is knowledge about other knowledge: what is known, how reliable it is, and which knowledge to use when.

Example: "Rules about fever are more reliable than rules about mood", or "For chest pain, consult the cardiology rules first."

Heuristic Knowledge

Heuristic knowledge is the knowledge of experts gained from experience — rules of thumb that usually work but are not guaranteed.

Example: "If the engine does not start and the lights are dim, check the battery first."

Structural Knowledge

Structural knowledge describes relationships between concepts and objects — such as kind-of, part-of and grouping. It is the basic knowledge needed for problem solving.

Example: "A wheel is part of a car", "A sparrow is a kind of bird".

Table 3.4: Types of knowledge compared

Type	Question it answers	Example
Declarative	What?	The Earth orbits the Sun
Procedural	How?	Algorithm to sort a list
Meta-knowledge	What do we know about our knowledge?	This rule is 80% reliable
Heuristic	What usually works?	Check the plug first
Structural	How are things related?	Dog is-a mammal; wheel part-of car

> **Memory Trick:** "D-P-M-H-S: Do People Make Happy Systems?" — Declarative, Procedural, Meta, Heuristic, Structural.

The AI Knowledge Cycle

An AI system uses knowledge inside a loop:

1. **Perception** — sensors take input from the environment (camera, keyboard, microphone).
2. **Learning** — useful patterns are extracted from the input.
3. **Knowledge representation and reasoning** — knowledge is stored and new conclusions are inferred.
4. **Planning** — a sequence of actions is decided.
5. **Execution** — actions are carried out, changing the environment, and the cycle repeats.

Properties of a Good Knowledge Representation System

A good KR system should have the following four classic properties (often asked as a 5-mark question).

Table 3.5: Properties of a good KR system

Property	Meaning	Why it matters
Representational adequacy	Can represent all kinds of knowledge needed in the domain	Nothing important is left out
Inferential adequacy	Can manipulate structures to derive new knowledge	Allows reasoning, not just lookup
Inferential efficiency	Can store extra guidance (e.g. indexes, hints) so inference is fast	Practical systems need speed

Property	Meaning	Why it matters
Acquisitional efficiency	Can acquire new knowledge easily, ideally automatically	Knowledge grows over time

Other desirable features:

- **Naturalness** — easy for humans to read and write.
- **Modularity** — knowledge can be added or removed in small pieces without rewriting everything.
- **Unambiguous semantics** — each sentence has one clear meaning.
- **Handling of incomplete and uncertain knowledge**.

> **Exam Tip:** When asked "properties of a good KR system", always write the four adequacy/efficiency properties with one line of meaning each. Examiners look for these exact four names.

Approaches (Techniques) of Knowledge Representation

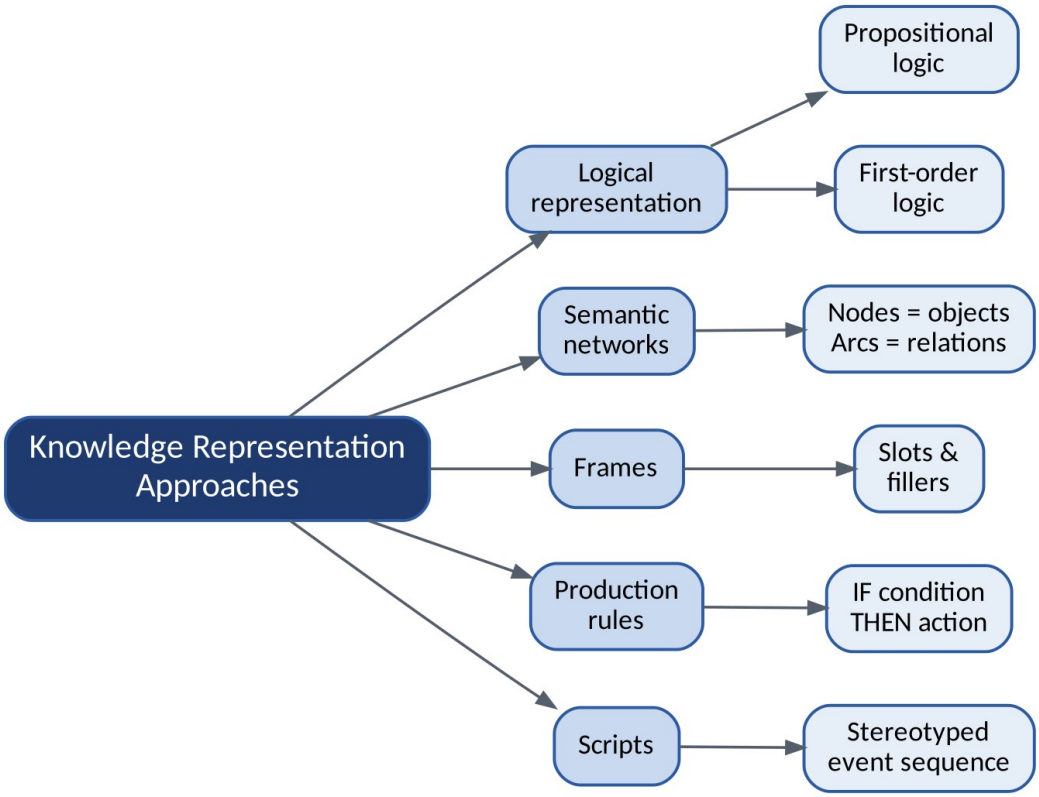


Figure 3.3: Main approaches to knowledge representation

There are four main techniques taught in almost every syllabus — **logical representation, semantic networks, frames and production rules** — plus **scripts**, which are frames specialised for events.

Logical Representation

Logical representation uses a formal language with precise syntax and semantics. Knowledge is written as sentences of **propositional logic** or **first-order (predicate) logic**, and conclusions are derived by sound inference rules.

Example:

```
Man(Socrates)
∀x Man(x) → Mortal(x)
Therefore: Mortal(Socrates)
```

Advantages:

- Precise and unambiguous; meaning is mathematically defined.
- Inference rules are sound, so derived conclusions are guaranteed true.
- Forms the basis of programming languages such as Prolog.

Disadvantages:

- Hard for non-experts to write.
- Inference can be slow for large knowledge bases.
- Pure logic handles uncertainty and exceptions poorly.

Propositional and first-order logic are studied in detail in the next chapters.

Semantic Networks

A **semantic network** (semantic net) represents knowledge as a **graph**:

- **Nodes** represent objects, concepts or values.
- **Labelled arcs (links)** represent relationships between them.

The two most important links are:

- **is-a** (subclass) — "Cat is-a Mammal".
- **instance-of** — "Tom is an instance of Cat".

Other links such as **has**, **part-of**, **owned-by** or **colour** describe properties.

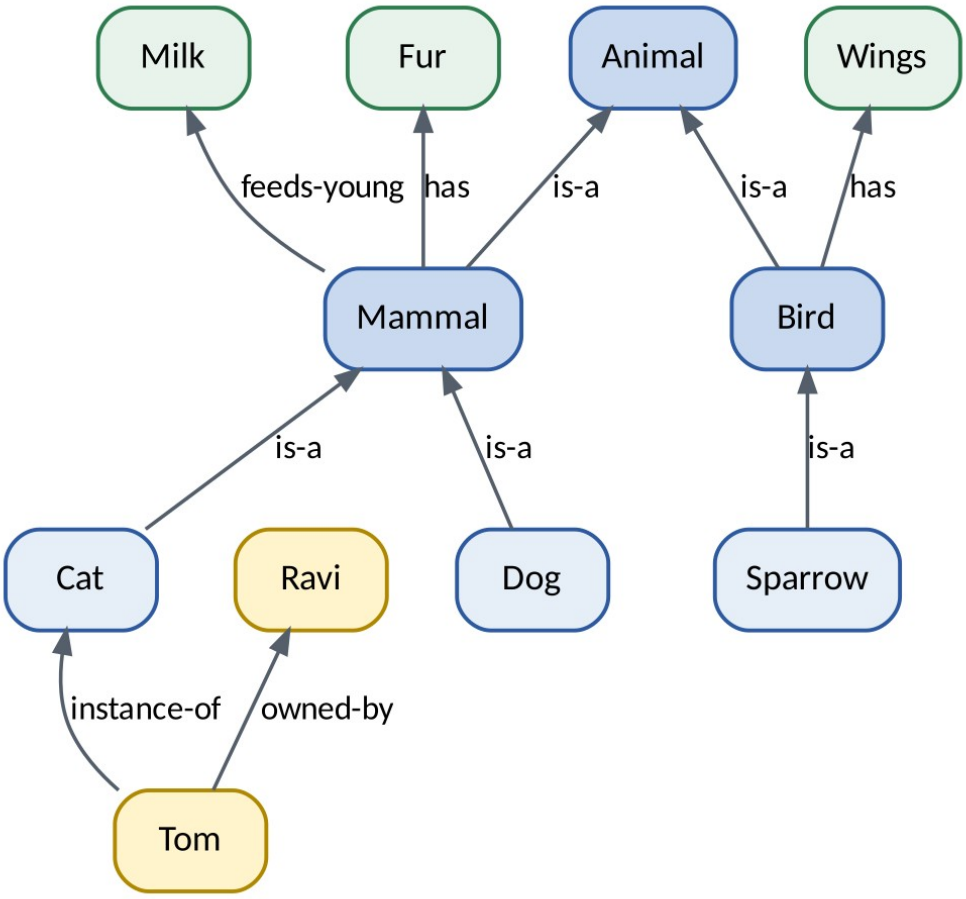


Figure 3.4: A semantic network about animals

Facts represented in Figure 3.4:

- Mammal is-a Animal; Bird is-a Animal.
- Cat and Dog are Mammals; Sparrow is a Bird.
- Mammals have fur and feed their young with milk; birds have wings.
- Tom is an instance of Cat and is owned by Ravi.

Inheritance: properties flow downward along is-a and instance-of links. To answer "Does Tom have fur?", the system follows Tom → Cat → Mammal and finds "has Fur". This saves storage because "has fur" is written once, not for every cat and dog.

Worked Example — Answering a query with a semantic net: Query: "Is Tom an animal?" Start at node Tom, follow instance-of to Cat, is-a to Mammal, is-a to Animal. A path exists, so the answer is YES. Query: "Does Tom have wings?" Following Tom → Cat → Mammal → Animal never reaches a "has Wings" link (that link belongs to Bird), so the answer is NO (or "not known" in an open-world system).

Advantages:

- Natural and easy to understand visually.
- Supports inheritance, so knowledge is stored compactly.
- Related knowledge is linked, so retrieval follows links quickly.

Disadvantages:

- No standard meaning for link names — different designers use them differently.

- Cannot easily express negation ("Tom is not a dog"), disjunction ("A or B") or quantifiers ("every", "some").
- Searching a very large network can be slow.

Frames

A **frame** (introduced by Marvin Minsky in 1974) is a record-like structure that describes a stereotyped object or situation. A frame is a collection of **slots**, and each slot holds a **filler** (value).

- **Slot** — an attribute (e.g. colour, legs, can-fly).
- **Filler** — the value of the slot (e.g. 2, yes).
- **Facets** — extra information about a slot: default value, type, range, and **procedural attachments** (demons) such as IF-NEEDED (compute the value when asked) and IF-ADDED (run when a value is stored).

Frames are linked by is-a and instance-of slots, so frames also support **inheritance**, including **default values** that can be **overridden**.

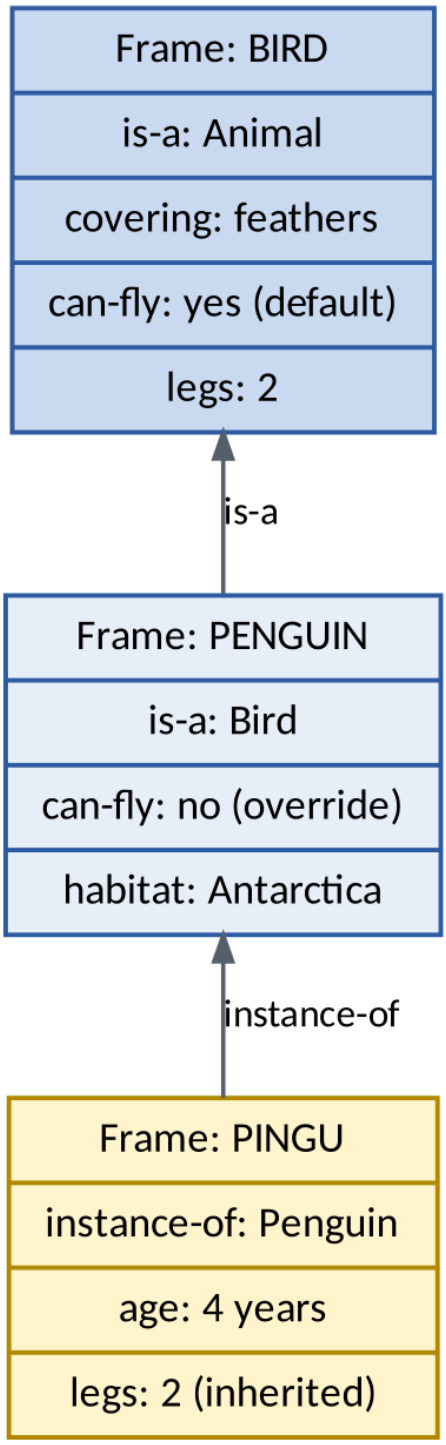


Figure 3.5: Frame hierarchy with inheritance and an overridden default

In Figure 3.5, the BIRD frame says birds can fly *by default*. The PENGUIN frame overrides this with "can-fly: no". PINGU, an instance of Penguin, inherits "legs: 2" from BIRD and "can-fly: no" from PENGUIN.

A frame written in text form:

```
(Frame  STUDENT
   is-a        : Person
   name        : (string)
   roll-no     : (integer)
```

```
   course      : B.C.A.
   semester    : default 5
   age         : IF-NEEDED compute from date-of-birth)
```

Advantages:

- Groups all knowledge about one object in one place.
- Default values and overriding model common-sense knowledge well.
- Procedural attachments allow computation inside the structure.
- Frames are the ancestors of classes in object-oriented programming.

Disadvantages:

- Inference with frames is less formal than logic.
- Handling exceptions and multiple inheritance can become messy.
- Not well suited to expressing general rules.

Production Rules

A **production rule** has the form

```
IF <condition>  THEN <action / conclusion>
```

A **production system** has three parts:

- **Set of rules** (rule base / long-term memory).
- **Working memory** (current facts / short-term memory).
- **Recognise–act cycle** (control strategy): find rules whose conditions match working memory, choose one (conflict resolution), fire it, update working memory, repeat.

Example rules for a traffic signal:

```
R1: IF signal is red              THEN stop
R2: IF signal is yellow           THEN slow down
R3: IF signal is green AND road clear THEN go
```

Advantages:

- Very natural — rules are close to how experts explain decisions.
- Modular: rules can be added or deleted independently.
- Easy to explain reasoning by listing the rules that fired.

Disadvantages:

- Large rule sets become hard to maintain and may conflict.
- Inefficient when many rules must be matched each cycle (solved by the Rete algorithm in CLIPS).
- Rules usually do not learn by themselves.

Production rules are the basis of forward chaining, backward chaining and CLIPS, all covered later in this unit.

Scripts

A **script** (Schank and Abelson, 1977) is a frame-like structure that represents a **stereotyped sequence of events** in a particular context. The classic example is the "restaurant script".

Table 3.6: Components of a script

Script component	Restaurant example
Entry conditions	Customer is hungry and has money
Props	Tables, menu, food, bill
Roles	Customer, waiter, cook, cashier
Track	Fast-food restaurant / fine dining
Scenes	Entering, ordering, eating, paying, leaving
Results	Customer is not hungry; has less money

With the script, the system can answer "Did Ravi pay?" from the story "Ravi went to a restaurant and ordered dosa" — because paying is an expected scene of the restaurant script.

Comparison of KR Approaches

Table 3.7: Comparison of knowledge representation approaches

Feature	Logic	Semantic net	Frame	Production rule
Basic unit	Sentence	Node and link	Slot–filler record	IF–THEN rule
Readability	Low for beginners	High (graphical)	High	High
Inheritance	Via rules	Yes (is-a links)	Yes, with defaults	No
Formal semantics	Yes, precise	Weak	Weak	Moderate
Quantifiers / negation	Yes	Difficult	Difficult	Limited
Typical tool	Prolog	Conceptual graphs	KL-ONE, OOP	CLIPS, OPS5

Exam Tip: For a 10-mark question "Explain knowledge representation techniques", draw one semantic net and one frame (simple boxes are enough), write two production rules and one logic sentence, and finish with the comparison table. Diagrams earn marks quickly.

Issues in Knowledge Representation

- **Important attributes** — attributes such as instance-of and is-a that occur in almost every domain need special support.
- **Relationships among attributes** — inverse relations (owns / owned-by), single-valued vs multi-valued attributes.
- **Choosing the granularity** — should "Ravi fed the dog" be stored as one fact or broken into primitives?
- **Representing sets of objects** — "All elephants are grey" vs "Elephants are many".
- **Finding the right structure** — selecting the relevant frame or script from a huge store.
- **The frame problem** — representing what does *not* change when an action happens.

Knowledge-Based Agent

A **knowledge-based agent** is an agent that keeps an internal **knowledge base (KB)** and uses **inference** to decide its actions.

- The **knowledge base** is a set of **sentences** written in a knowledge representation language. Sentences given at the start (not derived) are called **axioms**.

- The **inference engine** derives new sentences from old ones.

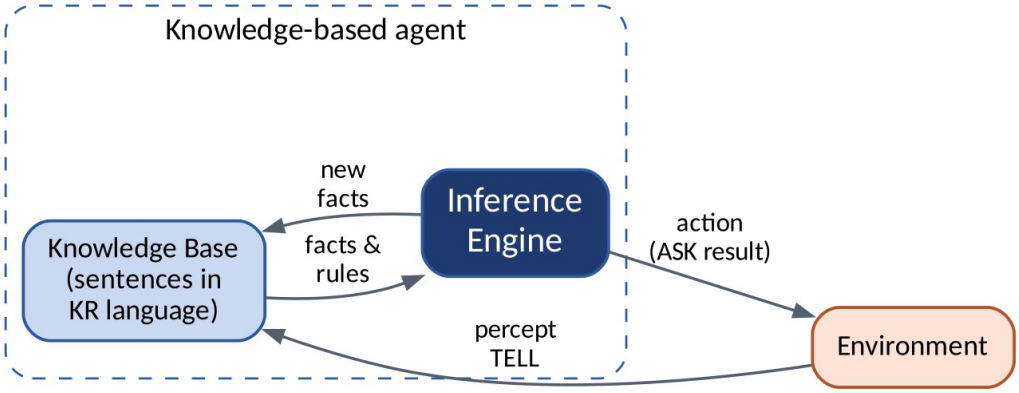


Figure 3.6: Architecture of a knowledge-based agent

TELL and ASK

The agent interacts with its KB using two standard operations:

- **TELL** — add a new sentence to the KB (e.g. what was perceived).
- **ASK** — query the KB about what to do; the answer must follow from what has been TELLed.

A generic knowledge-based agent (Russell and Norvig):

```
function KB-AGENT(percept) returns an action
   persistent: KB, a knowledge base
               t, a counter indicating time, initially 0

   TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
   action ← ASK(KB, MAKE-ACTION-QUERY(t))
   TELL(KB, MAKE-ACTION-SENTENCE(action, t))
   t ← t + 1
   return action
```

Each time the agent is called it does three things: (1) TELLs the KB what it perceives, (2) ASKs the KB what action to perform, (3) TELLs the KB which action it chose.

Levels of a Knowledge-Based Agent

Table 3.8: Three levels of a knowledge-based agent

Level	Description	Example (automated taxi)
Knowledge level	What the agent knows and its goals	Knows the bridge connects area A to area B
Logical level	How knowledge is encoded as sentences	Links(BridgeX, AreaA, AreaB)
Implementation level	Physical data structures and code	Strings, arrays, linked lists in memory

Declarative vs Procedural Approach to Building an Agent

- **Declarative approach** — start with an empty KB and TELL the agent sentences one by one until it knows how to act.
- **Procedural approach** — encode the desired behaviour directly as program code.
- Successful agents combine both.

Key Point: TELL puts knowledge in, ASK gets answers out, and the inference engine guarantees that ASK only returns answers that follow logically from what was TELLed.

Summary

- Knowledge representation stores knowledge in a formal form so that a program can reason with it; it has syntax and semantics.
- Types of knowledge: declarative, procedural, meta, heuristic and structural.
- A good KR system has representational adequacy, inferential adequacy, inferential efficiency and acquisitional efficiency.
- Main approaches: logic (precise), semantic networks (graphs with is-a inheritance), frames (slots, fillers, defaults), production rules (IF–THEN) and scripts (event sequences).
- A knowledge-based agent = KB + inference engine, operated by TELL and ASK, described at knowledge, logical and implementation levels.

Inference and Logical Reasoning

What is Inference?

Inference is the process of deriving new sentences (conclusions) from existing sentences (premises) in a knowledge base. The program component that does this is the **inference engine**.

Example:

```
Premise 1: If it rains, the road is wet.     (Rain → Wet)
Premise 2: It is raining.                    (Rain)
Conclusion: The road is wet.                 (Wet)
```

The conclusion "Wet" was not stored in the KB; it was **inferred**.

Basic Terms: Models, Entailment and Validity

Before studying inference methods we need four precise terms.

- **Model** — a possible world, i.e. one assignment of true/false (or objects and relations) that fixes the truth of every sentence. If sentence α is true in model m, we say m **satisfies** α, or m is a model of α.
- **Entailment** — KB **entails** α, written **KB ⊨ α**, if α is true in every model in which KB is true. In words: "α follows logically from KB".
- **Derivation** — **KB ⊢i α** means that α can be derived from KB by inference procedure i. Entailment is about meaning; derivation is about symbol manipulation.
- **Logical equivalence** — α ≡ β if both are true in exactly the same models.

Worked Example — Entailment: Let KB = {P, P → Q}. There are four models for P and Q. KB is true only in the model P = true, Q = true. In that model Q is true, so KB ⊨ Q. However KB does not entail ¬Q, because ¬Q is false in that model.

Soundness and Completeness

An inference procedure is judged by two properties.

Table 3.9: Soundness and completeness

Property	Definition	Meaning in plain words
Soundness (truth-preserving)	If KB ⊢ α then KB ⊨ α	Everything it derives is really true
Completeness	If KB ⊨ α then KB ⊢ α	It can derive everything that is really true

- A sound but incomplete procedure never lies but may miss some answers. Modus ponens alone is sound but not complete for general propositional logic.
- A complete but unsound procedure finds every true sentence but may also "prove" false ones — useless in practice.
- **Resolution** is sound and **refutation-complete** for propositional logic and for first-order logic.
- **Forward and backward chaining** are sound and complete for knowledge bases of **Horn clauses** (definite clauses).

Memory Trick: "Sound = Safe (no false answers). Complete = Catches all (no missed answers)." Think of a fishing net: a sound net catches only fish, a complete net catches every fish.

Types of Inference

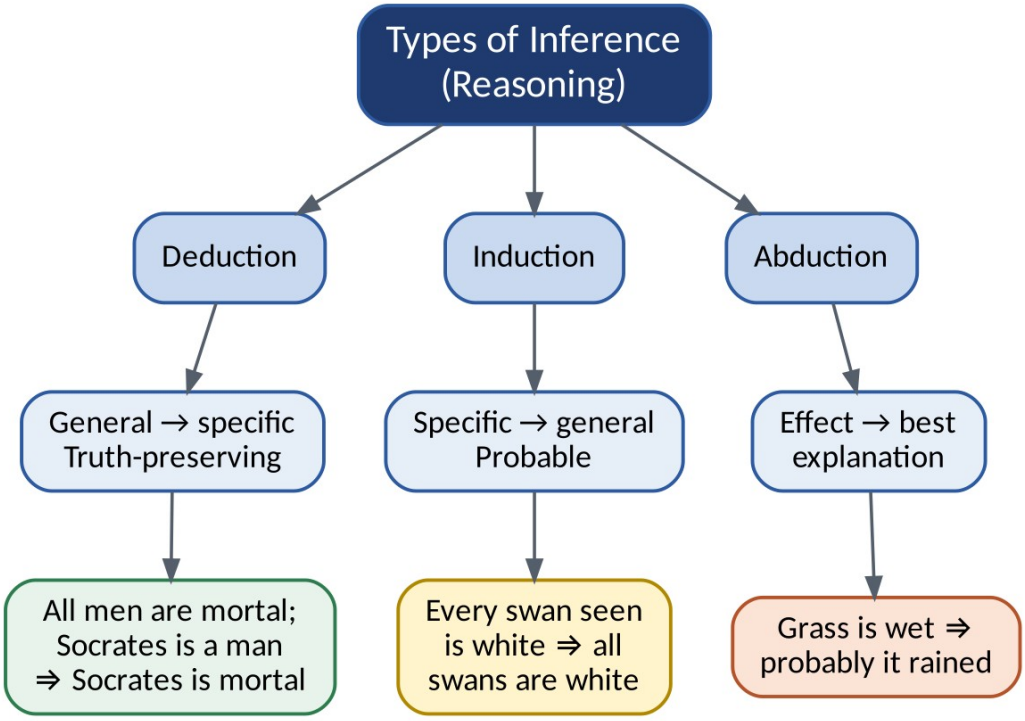


Figure 3.7: Deduction, induction and abduction

Deduction (Deductive Reasoning)

Deduction goes from general rules to a specific conclusion. If the premises are true and the rules are valid, the conclusion is **guaranteed** to be true. It is truth-preserving.

```
All humans are mortal.          ∀x Human(x) → Mortal(x)
Socrates is a human.            Human(Socrates)
∴ Socrates is mortal.           Mortal(Socrates)
```

Logical inference rules (modus ponens, resolution) are all deductive.

Induction (Inductive Reasoning)

Induction goes from specific observations to a general rule. The conclusion is **probable**, not certain; a single counter-example can destroy it.

```
Swan 1 is white. Swan 2 is white. ... Swan 1000 is white.
∴ All swans are white.          (false: black swans exist in Australia)
```

Machine learning is largely inductive: it generalises from training examples.

Abduction (Abductive Reasoning)

Abduction goes from an observed effect to the **most likely explanation** (cause). It is "inference to the best explanation" and is not guaranteed.

```
Rule:        If it rains, the grass is wet.    Rain → WetGrass
Observation: The grass is wet.                 WetGrass
∴ Hypothesis: It probably rained.              Rain   (maybe a sprinkler!)
```

Medical diagnosis is abductive: symptoms are effects, and the doctor infers the most likely disease.

Table 3.10: Deduction vs induction vs abduction

Feature	Deduction	Induction	Abduction
Direction	General → specific	Specific → general	Effect → cause
Certainty	Certain (if premises true)	Probable	Plausible guess
Sound?	Yes	No	No
Example use	Theorem proving	Machine learning	Diagnosis, fault finding
Logical form	P, P → Q ⊢ Q	P(a), P(b), … ⊢ ∀x P(x)	Q, P → Q ⊢ P (fallacy in strict logic)

Exam Tip: Notice that abduction has the same form as the fallacy of "affirming the consequent" (Q and P → Q, therefore P). Mention this in a 5-mark answer: abduction is useful in practice but is not logically valid.

Logical Reasoning: Types

Logical reasoning is the general process of drawing conclusions from knowledge using logic. Besides deduction, induction and abduction, the following types are named in most AI texts.

Common-Sense Reasoning

Informal reasoning based on everyday experience, often using heuristics. Example: "If I put my hand in fire, it will burn." Hard for computers because the amount of common-sense knowledge is enormous.

Monotonic Reasoning

In **monotonic reasoning**, once a conclusion is derived it stays true even when new knowledge is added. Adding facts can only increase the set of conclusions, never reduce it. Classical propositional and first-order logic are monotonic.

```
If KB ⊨ α  then  KB ∪ {β} ⊨ α   for any new sentence β
```

Example: "Earth revolves around the Sun" stays true whatever new facts are added.

- Advantage: proofs never need to be revised.
- Disadvantage: cannot represent default assumptions or handle exceptions.

Non-Monotonic Reasoning

In **non-monotonic reasoning**, a conclusion may be **withdrawn** when new information arrives. It models default and common-sense reasoning.

```
Birds can fly. Pingu is a bird.     ⇒ Pingu can fly.
New fact: Pingu is a penguin.       ⇒ withdraw "Pingu can fly".
```

Default logic, circumscription and truth-maintenance systems implement non-monotonic reasoning. The frame in Figure 3.5 (penguin overrides can-fly) is a non-monotonic idea.

Table 3.11: Monotonic vs non-monotonic reasoning

Feature	Monotonic	Non-monotonic
Effect of new knowledge	Conclusions only grow	Old conclusions may be retracted
Logic used	Classical PL, FOL	Default logic, circumscription
Handles exceptions	No	Yes
Example	Mathematics proofs	"Birds fly" unless penguin

Probabilistic Reasoning

Uses probabilities to handle uncertainty, e.g. "There is a 70% chance of rain". Bayesian networks are the standard tool. (Studied in detail in later units/courses.)

Two Families of Inference Algorithms

Inference algorithms in logic fall into two broad families:

- **Model checking** — enumerate models (e.g. a truth table) and check that α is true wherever KB is true. Simple but exponential: n symbols give 2^n rows.
- **Theorem proving** — apply inference rules directly to sentences to build a proof, without listing models. Examples: natural deduction with modus ponens, resolution, forward chaining and backward chaining.

Table 3.12: Inference methods in this unit

Method	Works on	Complete for	Covered in
Truth table (model checking)	Propositional logic	Propositional logic	Propositional Logic chapter
Resolution	CNF clauses	PL and FOL (refutation)	PL and FOL chapters
Forward chaining	Definite (Horn) clauses	Horn KBs	Chaining chapter
Backward chaining	Definite (Horn) clauses	Horn KBs	Chaining chapter, Prolog

> **Key Point:** Logical inference used by AI systems is deductive and sound. Induction and abduction are important forms of reasoning too, but their conclusions are only probable and must be treated as hypotheses.

Summary

- Inference derives new sentences from a KB; KB ⊨ α means α is true in every model of KB.
- Sound: derives only true sentences. Complete: derives all true sentences.
- Deduction (general → specific, certain), induction (specific → general, probable), abduction (effect → cause, plausible).
- Monotonic reasoning never retracts conclusions; non-monotonic reasoning can.
- Inference is done by model checking (truth tables) or theorem proving (rules, resolution, chaining).

Propositional Logic

Introduction

Propositional logic (PL), also called **Boolean logic**, **sentential logic** or **zeroth-order logic**, is the simplest form of logic. Knowledge is written as **propositions** — declarative statements that are either **true** or **false**, but not both.

Table 3.13: Which sentences are propositions?

Sentence	Proposition?	Reason
"Jaipur is in Rajasthan."	Yes	It is true
"5 + 3 = 9."	Yes	It is false (but still a proposition)
"Close the door."	No	A command has no truth value
"What time is it?"	No	A question has no truth value
"x + 2 = 5."	No	Truth depends on x (open sentence)

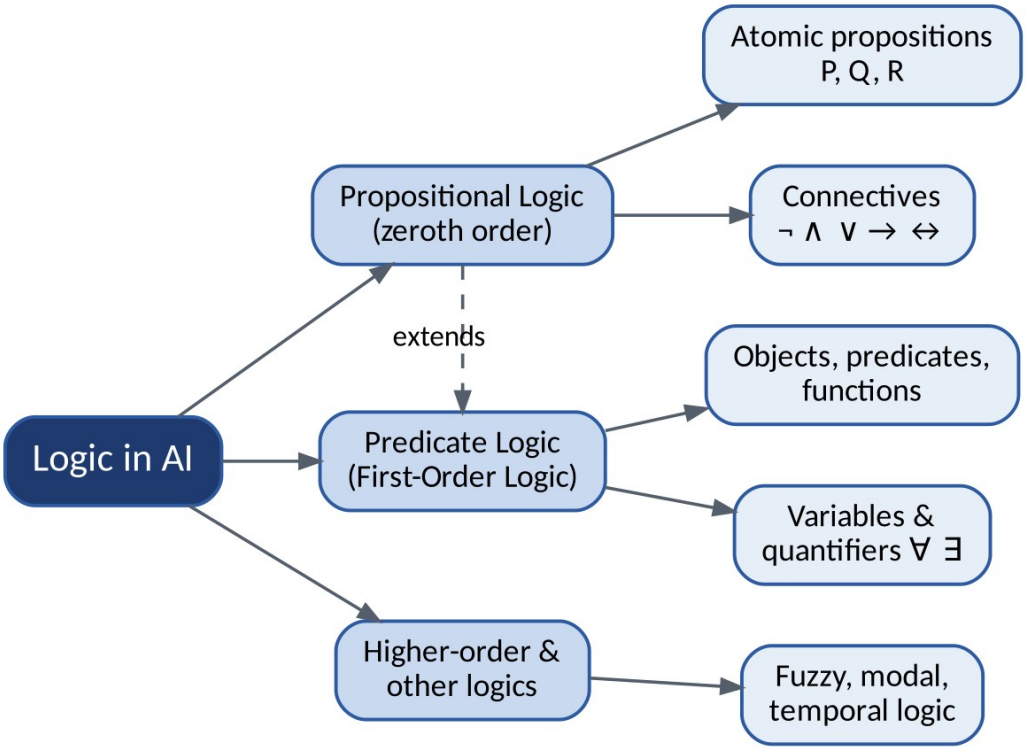


Figure 3.8: Propositional logic and first-order logic within logic in AI

Syntax of Propositional Logic

Atomic and Compound Propositions

- **Atomic proposition** — a single proposition symbol that cannot be broken down: P, Q, R, Rain, B11. The constants **True** and **False** are also atomic.
- **Compound (complex) proposition** — built from atomic propositions using **logical connectives** and parentheses: (P ∧ Q) → R.

Grammar (BNF) of Propositional Logic

```
Sentence        → AtomicSentence | ComplexSentence
AtomicSentence  → True | False | P | Q | R | ...
ComplexSentence → ( Sentence )
                | ¬ Sentence
                | Sentence ∧ Sentence
                | Sentence ∨ Sentence
                | Sentence → Sentence
                | Sentence ↔ Sentence
```

Logical Connectives

Table 3.14: The five logical connectives

Connective	Symbol	Name	Read as	Example
Negation	¬	NOT	"not P"	¬Rain
Conjunction	∧	AND	"P and Q"	Hot ∧ Humid
Disjunction	∨	OR (inclusive)	"P or Q"	Tea ∨ Coffee
Implication	→	IF–THEN (conditional)	"if P then Q"	Rain → Wet
Biconditional	↔	IF AND ONLY IF	"P iff Q"	Pass ↔ Marks ≥ 40

In an implication P → Q, P is called the **premise** (antecedent) and Q the **conclusion** (consequent). Implications are also called **rules** or **if–then statements**.

Semantics: Truth Tables for All Connectives

The **semantics** of PL says how to compute the truth value of any sentence from the truth values of its symbols. This is done with **truth tables**.

Negation

Table 3.15: Truth table of negation

P	¬P
T	F
F	T

Conjunction, Disjunction, Implication and Biconditional

Table 3.16: Truth tables of the binary connectives

P	Q	P ∧ Q	P ∨ Q	P → Q	P ↔ Q
T	T	T	T	T	T
T	F	F	T	F	F
F	T	F	T	T	F
F	F	F	F	T	T

Points to note:

- **P ∧ Q** is true only when both are true.
- **P ∨ Q** is false only when both are false (inclusive OR).
- **P → Q** is false **only** when P is true and Q is false. When P is false, the implication is true whatever Q is ("vacuously true"). Example: "If I win the lottery, I will buy you a car" is not broken if I never win.

- **P ↔ Q** is true when P and Q have the **same** value.

Related forms of an implication P → Q:

Table 3.17: Converse, inverse and contrapositive

Form	Sentence	Equivalent to P → Q?
Converse	Q → P	No
Inverse	¬P → ¬Q	No
Contrapositive	¬Q → ¬P	Yes

Memory Trick: "Implication is only broken when a TRUE promise gives a FALSE result" — T → F is the only false row. And "the Contrapositive is the only Cousin you can trust".

Operator Precedence

When parentheses are missing, connectives are applied in this order (highest first):

Table 3.18: Precedence of logical connectives

Precedence	Connective
1 (highest)	( ) parentheses
2	¬ negation
3	∧ conjunction
4	∨ disjunction
5	→ implication
6 (lowest)	↔ biconditional

Example: ¬P ∨ Q ∧ R → S is read as ((¬P) ∨ (Q ∧ R)) → S.

Tautology, Contradiction and Contingency

- **Tautology (valid sentence)** — true in **every** model. Example: P ∨ ¬P.
- **Contradiction (unsatisfiable sentence)** — false in **every** model. Example: P ∧ ¬P.
- **Contingency** — true in some models and false in others. Example: P → Q.

Worked Example — Prove (P → Q) ∧ P → Q is a tautology: Build the truth table. Rows (P, Q): (T,T): P → Q = T, (P → Q) ∧ P = T, whole = T. (T,F): P → Q = F, conjunction = F, whole = T. (F,T): P → Q = T, conjunction = F, whole = T. (F,F): P → Q = T, conjunction = F, whole = T. The last column is T in all four rows, so the sentence is a tautology. This is exactly the rule modus ponens written as one sentence.

A larger example — the **hypothetical syllogism** written as a sentence:

Table 3.19: ((P → Q) ∧ (Q → R)) → (P → R) is a tautology

P	Q	R	P → Q	Q → R	(P→Q) ∧ (Q→R)	P → R	Whole sentence
T	T	T	T	T	T	T	T
T	T	F	T	F	F	F	T
T	F	T	F	T	F	T	T
T	F	F	F	T	F	F	T

P	Q	R	P → Q	Q → R	(P→Q) ∧ (Q→R)	P → R	Whole sentence
F	T	T	T	T	T	T	T
F	T	F	T	F	F	T	T
F	F	T	T	T	T	T	T
F	F	F	T	T	T	T	T

Validity and Satisfiability

Table 3.20: Validity and satisfiability

Term	Definition	Example
Valid	True in all models (tautology)	P ∨ ¬P
Satisfiable	True in at least one model	P ∧ Q (true when both T)
Unsatisfiable	True in no model (contradiction)	P ∧ ¬P
Falsifiable	False in at least one model	P → Q

Two very important connections:

- α is **valid** if and only if **¬α is unsatisfiable**.
- **KB ⊨ α** if and only if **(KB ∧ ¬α) is unsatisfiable**. This is the basis of **proof by contradiction** (refutation), which resolution uses.

The problem of deciding whether a propositional sentence is satisfiable is called **SAT**; it was the first problem proved NP-complete (Cook, 1971).

Logical Equivalence Laws

Two sentences are **logically equivalent** (α ≡ β) if they have identical truth tables. The following laws are used to simplify sentences and to convert them to CNF.

Table 3.21: Laws of logical equivalence

Law	Equivalence
Identity	P ∧ True ≡ P; P ∨ False ≡ P
Domination	P ∨ True ≡ True; P ∧ False ≡ False
Idempotent	P ∧ P ≡ P; P ∨ P ≡ P
Double negation	¬(¬P) ≡ P
Commutative	P ∧ Q ≡ Q ∧ P; P ∨ Q ≡ Q ∨ P
Associative	(P ∧ Q) ∧ R ≡ P ∧ (Q ∧ R); same for ∨
Distributive	P ∧ (Q ∨ R) ≡ (P ∧ Q) ∨ (P ∧ R)
Distributive	P ∨ (Q ∧ R) ≡ (P ∨ Q) ∧ (P ∨ R)
De Morgan	¬(P ∧ Q) ≡ ¬P ∨ ¬Q; ¬(P ∨ Q) ≡ ¬P ∧ ¬Q
Absorption	P ∨ (P ∧ Q) ≡ P; P ∧ (P ∨ Q) ≡ P
Complement (negation)	P ∨ ¬P ≡ True; P ∧ ¬P ≡ False
Implication elimination	P → Q ≡ ¬P ∨ Q

Law	Equivalence
Contraposition	$P \to Q \equiv \neg Q \to \neg P$
Biconditional elimination	$P \leftrightarrow Q \equiv (P \to Q) \land (Q \to P)$

Worked Example — Simplify ¬(P ∨ ¬Q) ∨ (¬P ∧ ¬Q): By De Morgan, ¬(P ∨ ¬Q) ≡ ¬P ∧ Q. So the sentence is (¬P ∧ Q) ∨ (¬P ∧ ¬Q). By the distributive law (in reverse) this is ¬P ∧ (Q ∨ ¬Q). By the complement law Q ∨ ¬Q ≡ True, and by identity ¬P ∧ True ≡ ¬P. Final answer: ¬P.

Rules of Inference

An **inference rule** is a pattern that lets us derive a conclusion (below the line) from premises (above the line). All rules below are **sound**.

Table 3.22: Rules of inference in propositional logic

Rule	Premises	Conclusion
Modus Ponens	$P \to Q$, P	Q
Modus Tollens	$P \to Q$, $\neg Q$	$\neg P$
Hypothetical Syllogism	$P \to Q$, $Q \to R$	$P \to R$
Disjunctive Syllogism	$P \lor Q$, $\neg P$	Q
And-Elimination (Simplification)	$P \land Q$	P (and also Q)
And-Introduction (Conjunction)	P, Q	$P \land Q$
Addition (Or-Introduction)	P	$P \lor Q$
Double-negation elimination	$\neg\neg P$	P
Resolution	$P \lor Q$, $\neg P \lor R$	$Q \lor R$
Unit Resolution	$P \lor Q$, $\neg Q$	P

Plain-English examples:

- **Modus ponens:** "If it rains, the match is cancelled. It is raining. ∴ The match is cancelled."
- **Modus tollens:** "If it rains, the ground is wet. The ground is not wet. ∴ It did not rain."
- **Hypothetical syllogism:** "If I study, I pass. If I pass, I get a job. ∴ If I study, I get a job."
- **Disjunctive syllogism:** "The pen is in the bag or on the table. It is not in the bag. ∴ It is on the table."
- **And-elimination:** "Ravi is tall and smart. ∴ Ravi is tall."
- **Resolution:** "Ravi is in Jaipur or Delhi. Ravi is not in Jaipur or he is at JNU. ∴ Ravi is in Delhi or at JNU."

Worked Example — Natural deduction proof: KB: (1) P → Q, (2) Q → R, (3) ¬R. Prove ¬P. From (1) and (2) by hypothetical syllogism: (4) P → R. From (4) and (3) by modus tollens: (5) ¬P. Proved. Each step quotes a rule — always do this in exams.

Resolution: the Key Rule

Resolution takes two **clauses** that contain a **complementary pair of literals** (a literal and its negation) and produces a new clause, the **resolvent**, containing all remaining literals:

```
  l1 ∨ ... ∨ li ∨ ... ∨ lk        m1 ∨ ... ∨ mj ∨ ... ∨ mn
  ---------------------------------------------------------   where li = ¬mj
  l1 ∨ ... ∨ l(i-1) ∨ l(i+1) ∨ ... ∨ lk
     ∨ m1 ∨ ... ∨ m(j-1) ∨ m(j+1) ∨ ... ∨ mn
```

- A **literal** is an atom or a negated atom: P, ¬Q.
- A **clause** is a disjunction of literals: ¬P ∨ Q ∨ R.
- Duplicate literals in the result are removed (**factoring**): A ∨ A becomes A.
- Resolving P with ¬P gives the **empty clause** □, which means **contradiction**.

Resolution works only on sentences in **Conjunctive Normal Form**, so we first need CNF conversion.

Conjunctive Normal Form (CNF)

A sentence is in **CNF** if it is a **conjunction (AND) of clauses**, where each clause is a **disjunction (OR) of literals**.

```
(¬A ∨ B ∨ C) ∧ (¬B ∨ D) ∧ (E)          ← CNF
```

The other normal form, **Disjunctive Normal Form (DNF)**, is an OR of ANDs: (A ∧ B) ∨ (¬C ∧ D). Every propositional sentence can be converted to an equivalent CNF sentence.

Steps to Convert to CNF

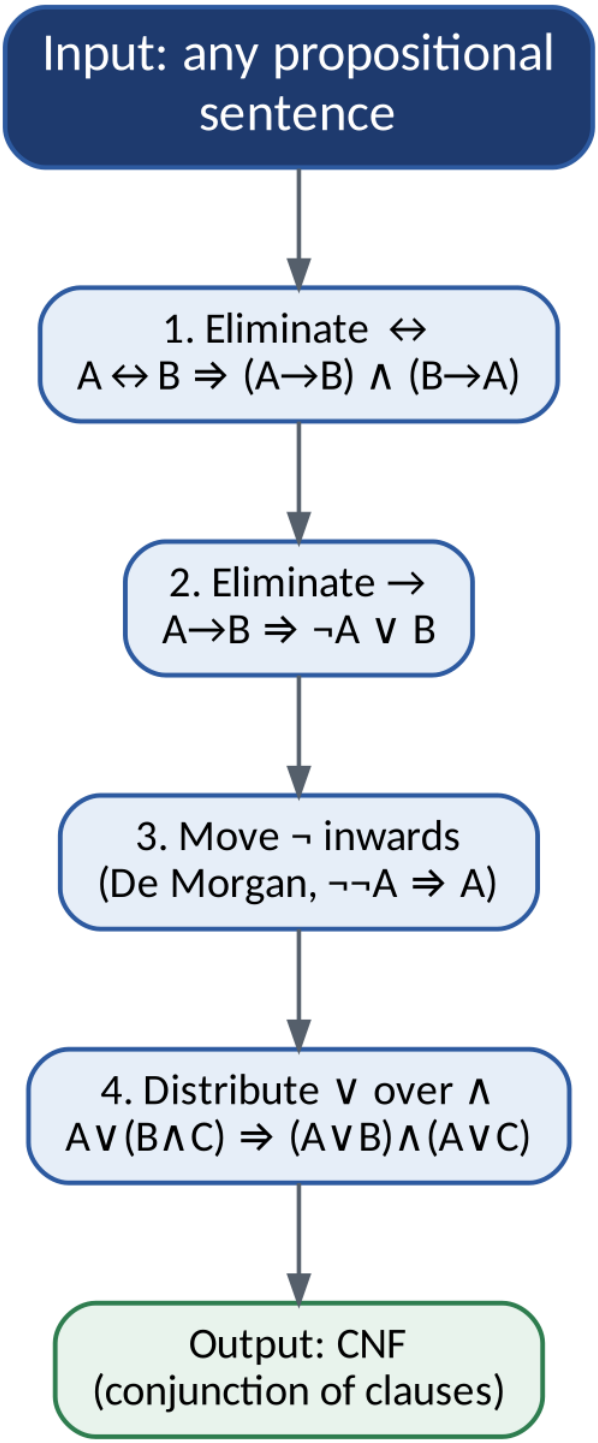


Figure 3.9: Four steps of CNF conversion

1. **Eliminate ↔ :** replace $\alpha \leftrightarrow \beta$ with $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$.
2. **Eliminate →:** replace $\alpha \rightarrow \beta$ with $\neg\alpha \vee \beta$.
3. **Move ¬ inwards** using De Morgan's laws and double negation: $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$; $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$; $\neg\neg\alpha \equiv \alpha$.
4. **Distribute ∨ over ∧:** $\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$. Flatten nested ANDs/ORs.

Worked CNF Examples

```
Example 1: Convert  B11 ↔ (P12 ∨ P21)  to CNF   (Wumpus world rule)

Step 1 (eliminate ↔):
   (B11 → (P12 ∨ P21)) ∧ ((P12 ∨ P21) → B11)
Step 2 (eliminate →):
   (¬B11 ∨ P12 ∨ P21) ∧ (¬(P12 ∨ P21) ∨ B11)
Step 3 (move ¬ inwards, De Morgan):
   (¬B11 ∨ P12 ∨ P21) ∧ ((¬P12 ∧ ¬P21) ∨ B11)
Step 4 (distribute ∨ over ∧):
   (¬B11 ∨ P12 ∨ P21) ∧ (¬P12 ∨ B11) ∧ (¬P21 ∨ B11)

Clauses: {¬B11, P12, P21}, {¬P12, B11}, {¬P21, B11}

Example 2: Convert  (P → Q) → R  to CNF

Step 2: ¬(¬P ∨ Q) ∨ R
Step 3: (¬¬P ∧ ¬Q) ∨ R   =   (P ∧ ¬Q) ∨ R
Step 4: (P ∨ R) ∧ (¬Q ∨ R)

Example 3: Convert  ¬(P ∨ Q) → (R ∧ S)  to CNF

Step 2: ¬¬(P ∨ Q) ∨ (R ∧ S)
Step 3: (P ∨ Q) ∨ (R ∧ S)
Step 4: (P ∨ Q ∨ R) ∧ (P ∨ Q ∨ S)
```

Exam Tip: Always show the four steps with their names, even if a step changes nothing ("Step 1: no ↔ present"). Most students lose marks by jumping straight to the answer. Check your final CNF on one row of a truth table if time permits.

Resolution Refutation (Proof by Contradiction)

To prove KB ⊨ α using resolution:

```
function PL-RESOLUTION(KB, α) returns true or false
   clauses ← the set of clauses in the CNF of (KB ∧ ¬α)
   new ← { }
   loop do
      for each pair of clauses Ci, Cj in clauses do
         resolvents ← PL-RESOLVE(Ci, Cj)
         if resolvents contains the empty clause then return true
         new ← new ∪ resolvents
      if new ⊆ clauses then return false      // nothing new: not provable
      clauses ← clauses ∪ new
```

In words:

1. Convert all KB sentences to CNF.
2. **Negate the goal** α and convert ¬α to CNF; add it to the clause set.
3. Repeatedly resolve pairs of clauses with complementary literals, adding resolvents.
4. If the **empty clause** □ is produced, the set is unsatisfiable, so KB ⊨ α (proved).
5. If no new clause can be produced, α does not follow.

Worked Example 1: A Simple Chain

KB: P → Q, Q → R, P. Prove R.

```
Clauses:  C1: ¬P ∨ Q    C2: ¬Q ∨ R    C3: P
Negated goal: C4: ¬R

C5 = resolve(C4, C2) on R :  ¬Q
```

```
C6 = resolve(C5, C1) on Q :  ¬P
C7 = resolve(C6, C3) on P :  □   (empty clause)
∴ R is proved.
```

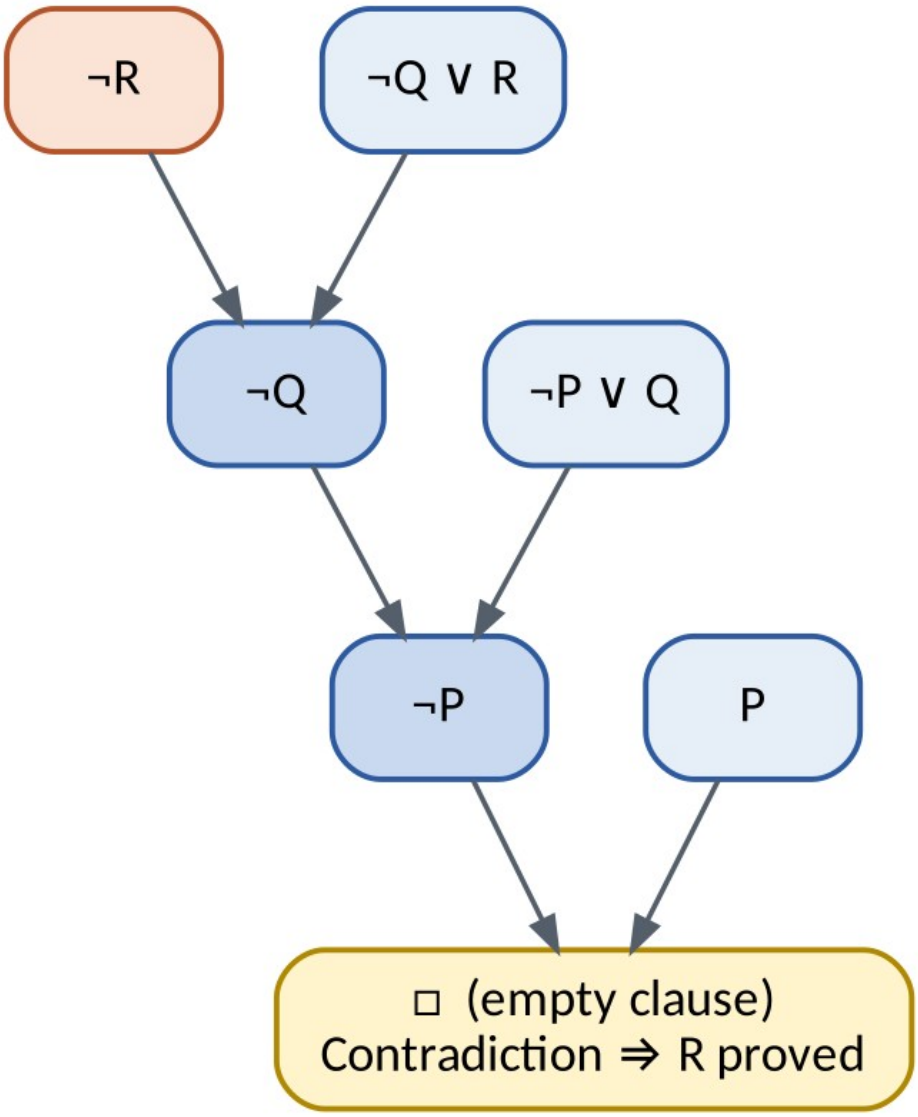


Figure 3.10: Resolution tree proving R

Worked Example 2: Hot, Humid and Rain

Worked Example — Will it rain? Facts: "If it is hot and humid, it will rain" (H ∧ U → R). "If it is humid, it is hot" (U → H). "It is humid" (U). Prove "It will rain" (R). CNF clauses: C1: ¬H ∨ ¬U ∨ R; C2: ¬U ∨ H; C3: U; negated goal C4: ¬R. Resolve C4 with C1 on R: C5 = ¬H ∨ ¬U. Resolve C5 with C2 on H: C6 = ¬U ∨ ¬U = ¬U (factoring). Resolve C6 with C3 on U: □. Contradiction, so R is proved.

Table 3.23: Resolution steps for the rain example

Step	Clause 1	Clause 2	Literal resolved	Resolvent
1	¬R	¬H ∨ ¬U ∨ R	R	¬H ∨ ¬U

Step	Clause 1	Clause 2	Literal resolved	Resolvent
2	¬H ∨ ¬U	¬U ∨ H	H	¬U
3	¬U	U	U	□

Properties of Propositional Resolution

- **Sound** — every resolvent is entailed by its parents.
- **Refutation-complete** — if KB ∧ ¬α is unsatisfiable, resolution will eventually derive □.
- **Terminates** in propositional logic because only finitely many clauses can be built from finitely many symbols.

Horn Clauses and Definite Clauses

Many practical knowledge bases use a restricted clause form that allows faster inference.

- **Definite clause** — a clause with **exactly one positive literal**: ¬A ∨ ¬B ∨ C, which is the same as A ∧ B → C.
- **Horn clause** — a clause with **at most one positive literal**. Includes definite clauses, facts (single positive literal, e.g. C) and goal clauses (no positive literal, e.g. ¬A ∨ ¬B).

Table 3.24: Horn and definite clauses

Clause	Positive literals	Type	Rule form
¬A ∨ ¬B ∨ C	1	Definite (Horn)	A ∧ B → C
C	1	Fact (Horn)	True → C
¬A ∨ ¬B	0	Goal clause (Horn)	A ∧ B → False
A ∨ B	2	Not Horn	—

Why Horn clauses matter:

- Every definite clause can be written as an IF–THEN rule — natural for humans.
- Inference can be done by **forward chaining** or **backward chaining**, in time **linear** in the size of the KB.
- Prolog programs are sets of definite clauses.

The Wumpus World: A Brief Example

The **Wumpus world** (Russell and Norvig) is a 4×4 cave. The agent starts in square [1,1]. Some squares contain **pits**; one contains the **Wumpus** (a monster); one contains **gold**. The agent perceives a **breeze** in squares next to a pit and a **stench** next to the Wumpus.

Propositional symbols: **Pxy** = there is a pit in [x,y]; **Bxy** = there is a breeze in [x,y].

```
R1: ¬P11                          (no pit in the start square)
R2: B11 ↔ (P12 ∨ P21)             (breeze in [1,1] iff pit next to it)
R3: B21 ↔ (P11 ∨ P22 ∨ P31)
R4: ¬B11                          (percept: no breeze in [1,1])
R5: B21                           (percept: breeze in [2,1])
```

Worked Example — Is [1,2] safe? Prove ¬P12 by resolution: From the CNF of R2 (Example 1 above) we have the clause ¬P12 ∨ B11. R4 gives ¬B11. The negated goal is P12. Resolve P12 with ¬P12 ∨ B11 to get B11. Resolve B11 with ¬B11 to get □. Hence ¬P12: there is no pit in [1,2], so the agent can move there safely. In the same way ¬P21 can be proved using the clause ¬P21 ∨ B11.

Limitations of Propositional Logic

- **Cannot express generalisations compactly.** "All students like AI" needs a separate symbol for every student: LikesAI_Ravi, LikesAI_Meena, ...
- **No objects, relations or functions.** "Ravi is the brother of Amit" becomes one opaque symbol with no internal structure.
- **No quantifiers** (∀, ∃). "Some student failed" cannot be written directly.
- **Rule explosion.** In the Wumpus world, the breeze rule must be written separately for each of the 16 squares.

These limitations motivate **first-order (predicate) logic**, covered next.

> **Key Point:** Propositional logic is decidable and simple, and resolution is complete for it, but it talks only about whole facts. To talk about objects, their properties and "all" or "some", we need first-order logic.

Summary

- A proposition is a statement that is true or false; PL builds compound sentences with ¬, ∧, ∨, →, ↔.
- Truth tables define every connective; P → Q is false only when P is T and Q is F. Precedence: ¬, ∧, ∨, →, ↔.
- Tautology = always true; contradiction = always false; contingency = sometimes true.
- KB ⊨ α iff KB ∧ ¬α is unsatisfiable — the basis of resolution refutation.
- Inference rules: modus ponens, modus tollens, hypothetical and disjunctive syllogism, and-elimination, resolution.
- CNF in four steps: remove ↔, remove →, move ¬ inwards, distribute ∨ over ∧.
- Horn clauses allow efficient forward and backward chaining.
- PL cannot express objects, relations or quantifiers.

Predicate Logic (First-Order Logic)

Why First-Order Logic?

Propositional logic treats "Ravi is a student" as a single indivisible symbol. It cannot see that the sentence is about an **object** (Ravi) having a **property** (being a student), and it cannot say "**all** students" or "**some** student".

First-order logic (FOL), also called **predicate logic** or **first-order predicate calculus (FOPC)**, assumes that the world contains:

- **Objects** — people, numbers, houses, colours, Ravi, Jaipur, 7.
- **Relations** — properties of one object (Red, Student) or links between several (BrotherOf, GreaterThan).
- **Functions** — relations with exactly one value for each input (FatherOf, Plus, Square).

Table 3.25: Propositional logic vs first-order logic

Feature	Propositional logic	First-order logic
Basic unit	Proposition (true/false fact)	Objects, relations, functions
Variables	No	Yes (x, y, z)
Quantifiers	No	∀ (for all), ∃ (there exists)
"All men are mortal"	Needs one symbol per man	∀x Man(x) → Mortal(x)
Expressive power	Low	High
Decidability	Decidable	Semi-decidable
Inference	Truth tables, resolution	Unification, GMP, FOL resolution

"First-order" means that quantifiers range over **objects** only. In **higher-order logic** one may also quantify over relations and functions ("for every property P...").

Syntax of FOL: the Basic Elements

Table 3.26: Basic elements of first-order logic

Element	Purpose	Examples
Constant symbols	Name specific objects	Ravi, Jaipur, 5, M1
Variables	Stand for any object	x, y, z, a, b
Predicate symbols	Properties and relations (return T/F)	Student(x), Loves(x, y)
Function symbols	Map objects to objects	Father(x), Plus(2, 3)
Connectives	Combine sentences	¬ ∧ ∨ → ↔
Quantifiers	Say "all" or "some"	∀ (universal), ∃ (existential)
Equality	Two terms refer to same object	Father(Ravi) = Mohan

Convention used in these notes: constants and predicates start with a capital letter (Ravi, Student); variables are lowercase (x, y).

Terms

A **term** is a logical expression that refers to an **object**:

- a constant: Ravi
- a variable: x
- a function applied to terms: Father(Ravi), LeftLeg(Father(x))

Functions do **not** return true/false; they return objects. Father(Ravi) is not a sentence; it names Ravi's father.

Atomic Sentences

An **atomic sentence** is a predicate applied to terms:

```
Student(Ravi)                  Ravi is a student
Brother(Amit, Ravi)            Amit is the brother of Ravi
Married(Father(Ravi), Mother(Ravi))
```

The **arity** of a predicate is the number of arguments: Student is unary (arity 1), Brother is binary (arity 2).

Complex Sentences

Atomic sentences are combined with connectives exactly as in PL:

```
Brother(Amit, Ravi) ∧ Student(Ravi)
¬Student(Mohan) → Teacher(Mohan)
```

Quantifiers

Universal Quantifier ∀

∀x P(x) is read "for all x, P(x)". It is true if P is true for **every** object in the domain.

```
All students are intelligent.
∀x Student(x) → Intelligent(x)
```

The main connective with ∀ is →. Writing ∀x Student(x) ∧ Intelligent(x) would wrongly say "everything in the universe is a student and is intelligent".

Existential Quantifier ∃

∃x P(x) is read "there exists an x such that P(x)". It is true if P is true for **at least one** object.

```
Some students are lazy.
∃x Student(x) ∧ Lazy(x)
```

The main connective with ∃ is ∧. Writing ∃x Student(x) → Lazy(x) is true even if there are no lazy students at all (any non-student x makes the implication true), so it says almost nothing.

> **Memory Trick:** "All-Arrow, Exists-And" — ∀ goes with →, ∃ goes with ∧. Say "A-A, E-A" before every translation.

Properties of Quantifiers

Table 3.27: Properties of quantifiers

Property	Equivalence
∀ commutes with ∀	∀x ∀y P(x, y) ≡ ∀y ∀x P(x, y)
∃ commutes with ∃	∃x ∃y P(x, y) ≡ ∃y ∃x P(x, y)
∀ and ∃ do NOT commute	∀x ∃y P(x, y) is not the same as ∃y ∀x P(x, y)

Property	Equivalence
De Morgan for ∀	¬∀x P(x) ≡ ∃x ¬P(x)
De Morgan for ∃	¬∃x P(x) ≡ ∀x ¬P(x)
Duality	∀x P(x) ≡ ¬∃x ¬P(x); ∃x P(x) ≡ ¬∀x ¬P(x)
∀ distributes over ∧	∀x (P(x) ∧ Q(x)) ≡ ∀x P(x) ∧ ∀x Q(x)
∃ distributes over ∨	∃x (P(x) ∨ Q(x)) ≡ ∃x P(x) ∨ ∃x Q(x)

Nested Quantifiers

When quantifiers are nested, **order matters**:

```
∀x ∃y Loves(x, y)     Everybody loves somebody.
                      (each person may love a different person)

∃y ∀x Loves(x, y)     There is someone whom everybody loves.
                      (one single person is loved by all)
```

Similarly, ∀x ∃y Mother(y, x) ("everyone has a mother") is true, but ∃y ∀x Mother(y, x) ("someone is the mother of everyone") is false.

Free and Bound Variables

- A variable is **bound** if it is within the scope of a quantifier that names it: in ∀x P(x, y), x is bound.
- A variable is **free** if it is not bound: y is free in ∀x P(x, y).
- A sentence with no free variables is a **closed formula** (well-formed sentence); only closed formulas have a definite truth value.

Equality

The **equality symbol** = states that two terms refer to the **same object**:

```
Father(Ravi) = Mohan          Ravi's father is Mohan.
```

Equality is needed to express counting:

```
Ravi has at least two sisters:
∃x ∃y Sister(x, Ravi) ∧ Sister(y, Ravi) ∧ ¬(x = y)

Ravi has exactly one father (uniqueness):
∃x Father(x, Ravi) ∧ ∀y (Father(y, Ravi) → y = x)
```

Without ¬(x = y), the first sentence would be satisfied by a single sister, because x and y could refer to the same person.

Translating English Sentences into FOL

Method

1. Identify the **objects** (constants), **properties/relations** (predicates) and **functions**.
2. Decide the quantifier: "all, every, each, any" → ∀; "some, a few, there is, at least one" → ∃; "no, none" → ∀ with ¬ (or ¬∃).
3. Use → under ∀ and ∧ under ∃.
4. For nested ideas, write the outer quantifier first.

Table 3.28: English to first-order logic translations

No.	English sentence	First-order logic
1	Ravi is a student.	Student(Ravi)
2	All students are intelligent.	∀x Student(x) → Intelligent(x)
3	Some students are lazy.	∃x Student(x) ∧ Lazy(x)
4	No student is perfect.	∀x Student(x) → ¬Perfect(x)
5	Not all birds can fly.	¬∀x (Bird(x) → Fly(x)), i.e. ∃x Bird(x) ∧ ¬Fly(x)
6	All cats like fish.	∀x Cat(x) → Likes(x, Fish)
7	Some boys play cricket.	∃x Boy(x) ∧ Plays(x, Cricket)
8	Every man respects his parents.	∀x ∀y Man(x) ∧ Parent(y, x) → Respects(x, y)
9	Everybody loves somebody.	∀x ∃y Loves(x, y)
10	There is someone whom everybody loves.	∃y ∀x Loves(x, y)
11	Brothers are siblings.	∀x ∀y Brother(x, y) → Sibling(x, y)
12	Sibling is a symmetric relation.	∀x ∀y Sibling(x, y) ↔ Sibling(y, x)
13	Ravi's mother is a doctor.	Doctor(Mother(Ravi))
14	A student who studies hard passes.	∀x Student(x) ∧ StudiesHard(x) → Passes(x)
15	Every number is either even or odd.	∀x Number(x) → Even(x) ∨ Odd(x)
16	Only one student failed in Maths.	∃x Student(x) ∧ Failed(x, Maths) ∧ ∀y (Student(y) ∧ Failed(y, Maths) → y = x)
17	Ravi has at least two sisters.	∃x ∃y Sister(x, Ravi) ∧ Sister(y, Ravi) ∧ ¬(x = y)
18	Every breezy square is next to a pit.	∀s Breezy(s) → ∃r Adjacent(r, s) ∧ Pit(r)
19	Whoever loves all animals is loved by someone.	∀x [∀y Animal(y) → Loves(x, y)] → [∃y Loves(y, x)]
20	Anyone who kills an animal is loved by no one.	∀x [∃y Animal(y) ∧ Kills(x, y)] → [∀z ¬Loves(z, x)]

Exam Tip: In translation questions, define your predicates first ("Student(x): x is a student; Loves(x, y): x loves y"). Examiners give marks for consistent naming even when the final formula has a small slip.

Worked Example — "Every student who takes AI likes some teacher": Objects: students, teachers, the course AI. Predicates: Student(x), Takes(x, c), Teacher(y), Likes(x, y). Outer idea is "every student who takes AI" → ∀ with →. Inner idea "likes some teacher" → ∃ with ∧. Answer: ∀x (Student(x) ∧ Takes(x, AI)) → ∃y (Teacher(y) ∧ Likes(x, y)).

Inference in First-Order Logic

FOL inference extends propositional inference with two extra ideas: **substitution** of terms for variables and **unification** to find the right substitution.

A **substitution** θ = {x/Ravi, y/Jaipur} means "replace x by Ravi and y by Jaipur". SUBST(θ, α) is the result of applying θ to sentence α.

```
SUBST({x/Ravi}, Student(x) → Intelligent(x))
   = Student(Ravi) → Intelligent(Ravi)
```

Universal Instantiation (UI)

From a universally quantified sentence we may infer **any** instance obtained by substituting a **ground term** (a term with no variables) for the variable:

```
  ∀v α
  ----------------------   for any ground term g
  SUBST({v/g}, α)
```

Example:

```
∀x King(x) ∧ Greedy(x) → Evil(x)
⇒ King(John) ∧ Greedy(John) → Evil(John)
⇒ King(Richard) ∧ Greedy(Richard) → Evil(Richard)
⇒ King(Father(John)) ∧ Greedy(Father(John)) → Evil(Father(John))
```

UI can be applied many times, producing many different sentences.

Existential Instantiation (EI)

From an existentially quantified sentence we may infer one instance using a **new constant symbol** that appears nowhere else in the KB (a **Skolem constant**):

```
  ∃v α
  ----------------------   k is a brand-new constant
  SUBST({v/k}, α)
```

Example:

```
∃x Crown(x) ∧ OnHead(x, John)
⇒ Crown(C1) ∧ OnHead(C1, John)        (C1 is new)
```

EI is applied **only once**, and the original sentence is then discarded. We must not use an existing constant: from "someone killed the victim" we cannot conclude "Ravi killed the victim".

Table 3.29: Universal vs existential instantiation

Feature	Universal Instantiation	Existential Instantiation
Applies to	∀x α	∃x α
Replace variable by	Any ground term	A new Skolem constant
How many times	Any number	Exactly once
Original sentence	Kept	Discarded

Propositionalization: by applying UI with every ground term and EI once, a FOL KB can be turned into a propositional KB and solved by PL methods. This works but is wasteful (with functions there are infinitely many ground terms). Unification gives a smarter, "lifted" approach.

Unification

Unification is the process of finding a substitution that makes two logical expressions **identical**. The algorithm UNIFY takes two sentences p and q and returns a **unifier** θ such that SUBST(θ, p) = SUBST(θ, q), or **failure** if none exists.

Most General Unifier (MGU)

Two expressions may have many unifiers. For Knows(John, x) and Knows(y, z):

- θ1 = {y/John, x/z} gives Knows(John, z).
- θ2 = {y/John, x/John, z/John} gives Knows(John, John).

θ1 is **more general** because it makes fewer commitments; θ2 can be obtained from θ1 by a further substitution {z/John}. The **Most General Unifier (MGU)** is the unifier that places the fewest restrictions on the variables. For any unifiable pair the MGU is unique up to renaming of variables.

Conditions for Unification

- The **predicate symbols** must be the same (Knows cannot unify with Likes).
- The number of **arguments** (arity) must be the same.
- Two different **constants** cannot unify (John ≠ Jane).
- A variable can be replaced by a constant, another variable or a function term — **but not by a term that contains the same variable** (the **occurs check**: x cannot unify with f(x)).
- The same variable must not receive two different values.

Unification Algorithm

```
function UNIFY(x, y, θ) returns a substitution to make x and y identical
   if θ = failure then return failure
   else if x = y then return θ
   else if VARIABLE?(x) then return UNIFY-VAR(x, y, θ)
   else if VARIABLE?(y) then return UNIFY-VAR(y, x, θ)
   else if COMPOUND?(x) and COMPOUND?(y) then
        return UNIFY(ARGS(x), ARGS(y), UNIFY(OP(x), OP(y), θ))
   else if LIST?(x) and LIST?(y) then
        return UNIFY(REST(x), REST(y), UNIFY(FIRST(x), FIRST(y), θ))
   else return failure

function UNIFY-VAR(var, x, θ) returns a substitution
   if {var/val} ∈ θ then return UNIFY(val, x, θ)
   else if {x/val} ∈ θ then return UNIFY(var, val, θ)
   else if OCCUR-CHECK?(var, x) then return failure
   else return θ ∪ {var/x}
```

The algorithm scans the two expressions **left to right**, argument by argument, extending θ each time a variable meets a term.

Unification Worked Examples

Table 3.30: Unification examples and MGUs

Expression 1	Expression 2	MGU θ	Unified result
Knows(John, x)	Knows(John, Jane)	{x/Jane}	Knows(John, Jane)
Knows(John, x)	Knows(y, Bill)	{x/Bill, y/John}	Knows(John, Bill)
Knows(John, x)	Knows(y, Mother(y))	{y/John, x/Mother(John)}	Knows(John, Mother(John))

Expression 1	Expression 2	MGU θ	Unified result
Knows(John, x)	Knows(x, Elizabeth)	fail (x cannot be John and Elizabeth)	—
Knows(John, x)	Knows(x2, Elizabeth)	{x/Elizabeth, x2/John}	Knows(John, Elizabeth)
P(x, f(x))	P(a, y)	{x/a, y/f(a)}	P(a, f(a))
P(x, x)	P(a, b)	fail (a ≠ b)	—
P(x)	P(f(x))	fail (occurs check)	—
P(a, x)	Q(a, x)	fail (different predicates)	—
P(x, y)	P(y, a)	{x/a, y/a}	P(a, a)

The fourth row fails only because both sentences happen to use the same variable name x. Renaming the variables of one sentence (**standardizing apart**) fixes it, as in the fifth row.

Worked Example — Find the MGU of P(x, g(y), z) and P(f(a), w, w): Compare argument 1: x with f(a) gives {x/f(a)}. Argument 2: g(y) with w gives {w/g(y)}. Argument 3: z with w; w is already bound to g(y), so z must unify with g(y), giving {z/g(y)}. MGU θ = {x/f(a), w/g(y), z/g(y)}. Check: both expressions become P(f(a), g(y), g(y)). Unified.

Worked Example — Find the MGU of Q(a, x, f(g(y))) and Q(z, f(z), f(u)): Argument 1: a with z gives {z/a}. Argument 2: x with f(z); apply current θ so f(z) becomes f(a), giving {x/f(a)}. Argument 3: f(g(y)) with f(u); same function f, so unify g(y) with u, giving {u/g(y)}. MGU θ = {z/a, x/f(a), u/g(y)}. Both become Q(a, f(a), f(g(y))).

Generalized Modus Ponens (GMP)

Generalized Modus Ponens is the lifted version of modus ponens. For atomic sentences p1', ..., pn' and a rule (p1 ∧ ... ∧ pn → q), if there is a substitution θ such that SUBST(θ, pi') = SUBST(θ, pi) for all i, then:

```
  p1',  p2',  ...,  pn',   (p1 ∧ p2 ∧ ... ∧ pn → q)
  ------------------------------------------------
                  SUBST(θ, q)
```

Example:

```
p1' = King(John)          p1 = King(x)
p2' = Greedy(y)           p2 = Greedy(x)          (everyone is greedy)
Rule: King(x) ∧ Greedy(x) → Evil(x)

θ = {x/John, y/John}
Conclusion: SUBST(θ, Evil(x)) = Evil(John)
```

GMP is sound. It is used by forward chaining and backward chaining over first-order definite clauses (next chapter).

> **Key Point:** Unification is the engine of FOL inference. GMP, forward chaining, backward chaining, FOL resolution and Prolog all depend on unifying a fact or goal with the premise or conclusion of a rule.

Converting FOL Sentences to CNF

FOL resolution needs clauses, so FOL sentences are converted to CNF in these steps:

1. **Eliminate ↔ and →** (as in PL).
2. **Move ¬ inwards** — also using ¬∀x p ≡ ∃x ¬p and ¬∃x p ≡ ∀x ¬p.

3. **Standardize variables apart** — each quantifier gets its own variable name.
4. **Skolemize** — remove ∃. If the ∃ is not inside any ∀, replace its variable with a new **Skolem constant**. If it is inside ∀x, replace it with a **Skolem function** of x, e.g. F(x).
5. **Drop universal quantifiers** — all remaining variables are understood as universally quantified.
6. **Distribute ∨ over ∧** to get a conjunction of clauses.

```
Example: "Everyone who loves all animals is loved by someone."
∀x [∀y Animal(y) → Loves(x,y)] → [∃y Loves(y,x)]

1. ∀x ¬[∀y ¬Animal(y) ∨ Loves(x,y)] ∨ [∃y Loves(y,x)]
2. ∀x [∃y Animal(y) ∧ ¬Loves(x,y)] ∨ [∃y Loves(y,x)]
3. ∀x [∃y Animal(y) ∧ ¬Loves(x,y)] ∨ [∃z Loves(z,x)]
4. ∀x [Animal(F(x)) ∧ ¬Loves(x,F(x))] ∨ Loves(G(x),x)
5. [Animal(F(x)) ∧ ¬Loves(x,F(x))] ∨ Loves(G(x),x)
6. [Animal(F(x)) ∨ Loves(G(x),x)] ∧ [¬Loves(x,F(x)) ∨ Loves(G(x),x)]
```

Here F(x) is "the animal that x does not love" and G(x) is "the person who loves x". They are Skolem functions because each depends on x.

Resolution in First-Order Logic

The FOL resolution rule is propositional resolution plus unification. Two clauses that contain literals li and mj which **unify** (with MGU θ) after one is negated are resolved, and θ is applied to the resolvent:

```
  l1 ∨ ... ∨ lk,     m1 ∨ ... ∨ mn          UNIFY(li, ¬mj) = θ
  ---------------------------------------------------------------
  SUBST(θ, l1 ∨ ... ∨ l(i-1) ∨ l(i+1) ∨ ... ∨ lk
              ∨ m1 ∨ ... ∨ m(j-1) ∨ m(j+1) ∨ ... ∨ mn)
```

Example: resolve [Animal(F(x)) ∨ Loves(G(x), x)] with [¬Loves(u, v) ∨ ¬Kills(u, v)], unifying Loves(G(x), x) with Loves(u, v) by θ = {u/G(x), v/x}: resolvent Animal(F(x)) ∨ ¬Kills(G(x), x).

Procedure for a Resolution Proof

1. Translate all English sentences into FOL.
2. Convert every sentence into CNF clauses.
3. **Negate the goal** and convert it to clause form; add it to the set.
4. Resolve pairs of clauses, using unification, until the **empty clause** □ appears.
5. The empty clause shows a contradiction, so the original goal is proved.

Full Worked Proof: Colonel West is a Criminal

This is the standard Russell and Norvig example.

The story: "The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American." **Prove: West is a criminal.**

Step 1 — FOL sentences:

```
1. American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) → Criminal(x)
   ("It is a crime for an American to sell weapons to hostile nations")
2. ∃x Owns(Nono, x) ∧ Missile(x)
   ("Nono has some missiles")  → by EI with new constant M1:
   Owns(Nono, M1)   and   Missile(M1)
3. Missile(x) ∧ Owns(Nono, x) → Sells(West, x, Nono)
   ("All of its missiles were sold to it by Colonel West")
4. Missile(x) → Weapon(x)          ("Missiles are weapons")
```

```
5. Enemy(x, America) → Hostile(x)  ("An enemy of America is hostile")
6. American(West)                  ("West is American")
7. Enemy(Nono, America)            ("Nono is an enemy of America")
```

Sentences 4 and 5 are background knowledge that the story assumes.

Step 2 — Clauses (CNF):

```
C1: ¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x, y, z) ∨ ¬Hostile(z)
      ∨ Criminal(x)
C2: ¬Missile(x) ∨ ¬Owns(Nono, x) ∨ Sells(West, x, Nono)
C3: ¬Enemy(x, America) ∨ Hostile(x)
C4: ¬Missile(x) ∨ Weapon(x)
C5: Owns(Nono, M1)
C6: Missile(M1)
C7: American(West)
C8: Enemy(Nono, America)
```

Step 3 — Negated goal:

```
C9: ¬Criminal(West)
```

Step 4 — Resolution steps:

Table 3.31: Resolution proof that West is a criminal

Step	Resolve	Unifier	Resolvent
1	C9 with C1	{x/West}	¬American(West) ∨ ¬Weapon(y) ∨ ¬Sells(West,y,z) ∨ ¬Hostile(z)
2	Step 1 with C7	{ }	¬Weapon(y) ∨ ¬Sells(West,y,z) ∨ ¬Hostile(z)
3	Step 2 with C4	{x/y}	¬Missile(y) ∨ ¬Sells(West,y,z) ∨ ¬Hostile(z)
4	Step 3 with C6	{y/M1}	¬Sells(West,M1,z) ∨ ¬Hostile(z)
5	Step 4 with C2	{x/M1, z/Nono}	¬Missile(M1) ∨ ¬Owns(Nono,M1) ∨ ¬Hostile(Nono)
6	Step 5 with C6	{ }	¬Owns(Nono,M1) ∨ ¬Hostile(Nono)
7	Step 6 with C5	{ }	¬Hostile(Nono)
8	Step 7 with C3	{x/Nono}	¬Enemy(Nono, America)
9	Step 8 with C8	{ }	□ (empty clause)

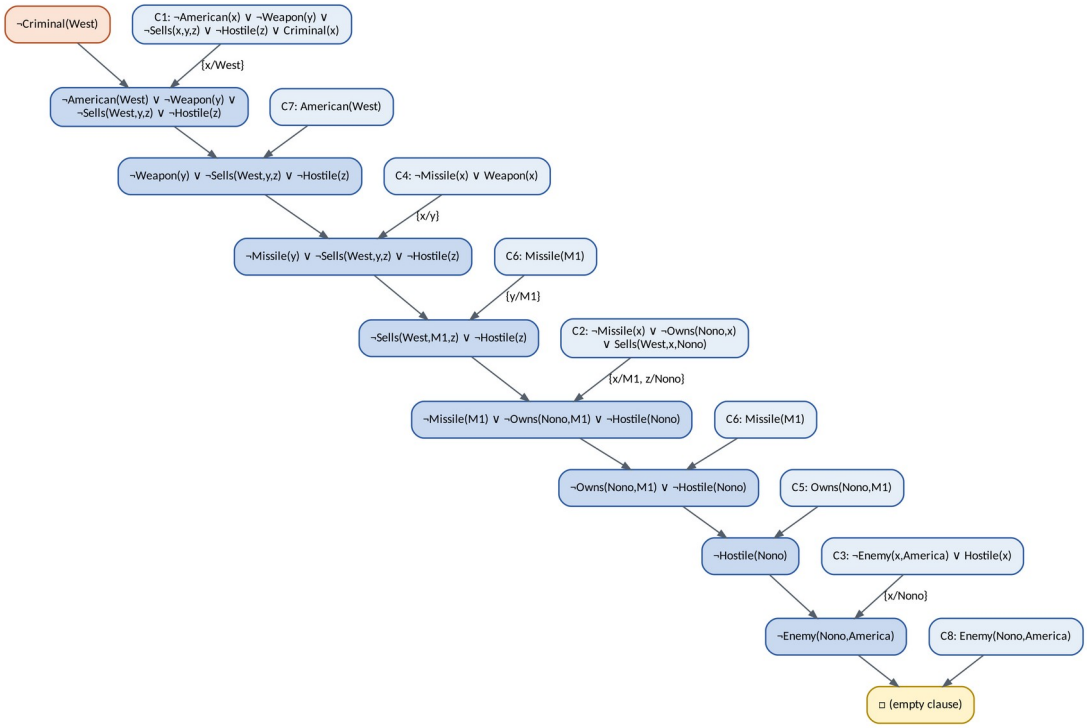


Figure 3.11: Resolution refutation tree for "West is a criminal"

The empty clause is derived, so the assumption ¬Criminal(West) leads to a contradiction. **Therefore Criminal(West) is true.**

Note that the variables of each clause are local to that clause. In step 3, the x of C4 is renamed to match y, and in step 5 the x of C2 is a fresh variable that unifies with M1.

> **Exam Tip:** In a resolution answer, write three clear parts with headings: (a) FOL sentences, (b) clauses numbered C1, C2, ..., (c) the proof table with the unifier in every row and the empty clause at the end. Draw the tree if time allows. A second classic story — "Marcus was a man ... Marcus hated Caesar" — is solved fully in the Important Questions chapter.

Properties of FOL Resolution

- **Sound** and **refutation-complete**: if a set of clauses is unsatisfiable, resolution will find a contradiction.
- FOL is only **semi-decidable**: if the goal is entailed, a proof will be found; if not, the procedure may run forever.
- Strategies to speed it up: **unit preference** (prefer resolving with single-literal clauses), **set of support** (always use the negated goal or its descendants), **input resolution** and **subsumption** (delete clauses that are more specific than existing ones).

Summary

- FOL describes objects, predicates (relations) and functions, with variables and quantifiers ∀ and ∃.
- ∀ is normally used with →, ∃ with ∧; nested quantifier order changes the meaning.
- ¬∀x P ≡ ∃x ¬P and ¬∃x P ≡ ∀x ¬P; equality is needed for counting and uniqueness.
- UI substitutes any ground term; EI substitutes one new Skolem constant.
- Unification finds the MGU; it fails on different constants, different predicates or the occurs check.

- GMP: from p1', ..., pn' and p1 ∧ ... ∧ pn → q, infer SUBST(θ, q).
- FOL CNF: remove →, move ¬, standardize, Skolemize, drop ∀, distribute.
- FOL resolution (with unification) proved "West is a criminal" in nine steps.

Forward Chaining and Backward Chaining

Introduction

Resolution can prove anything that follows from a knowledge base, but it is general and can be slow. Most practical **rule-based systems** and **expert systems** store knowledge as **IF–THEN rules** (definite clauses) and use two simpler, faster inference strategies:

- **Forward chaining** — start from the **known facts** and apply rules to derive new facts until the goal is reached. It is **data-driven**.
- **Backward chaining** — start from the **goal** and work backwards through rules to find facts that support it. It is **goal-driven**.

Both rely on **modus ponens** (in FOL, **Generalized Modus Ponens** with unification), and both are **sound** and **complete** for knowledge bases made of **definite clauses** (Horn clauses with exactly one positive literal).

The Shape of a Rule Base

```
Facts (atomic sentences):     A        Missile(M1)
Rules (definite clauses):     A ∧ B → L
                              Missile(x) → Weapon(x)
Goal (query):                 Q ?      Criminal(West) ?
```

The IF-part of a rule is called the **antecedent** (premise, body, LHS) and the THEN-part the **consequent** (conclusion, head, RHS).

> **Key Point:** Forward chaining moves from IF-parts to THEN-parts (facts → goal). Backward chaining moves from THEN-parts to IF-parts (goal → facts). The same rule base can be used both ways.

Forward Chaining

Idea

Forward chaining (forward reasoning, data-driven inference, bottom-up reasoning) starts with the facts in working memory. In each cycle it finds every rule whose premises are all satisfied by known facts, **fires** it, and adds its conclusion as a new fact. It stops when the goal appears or when no rule can add anything new (a **fixed point**).

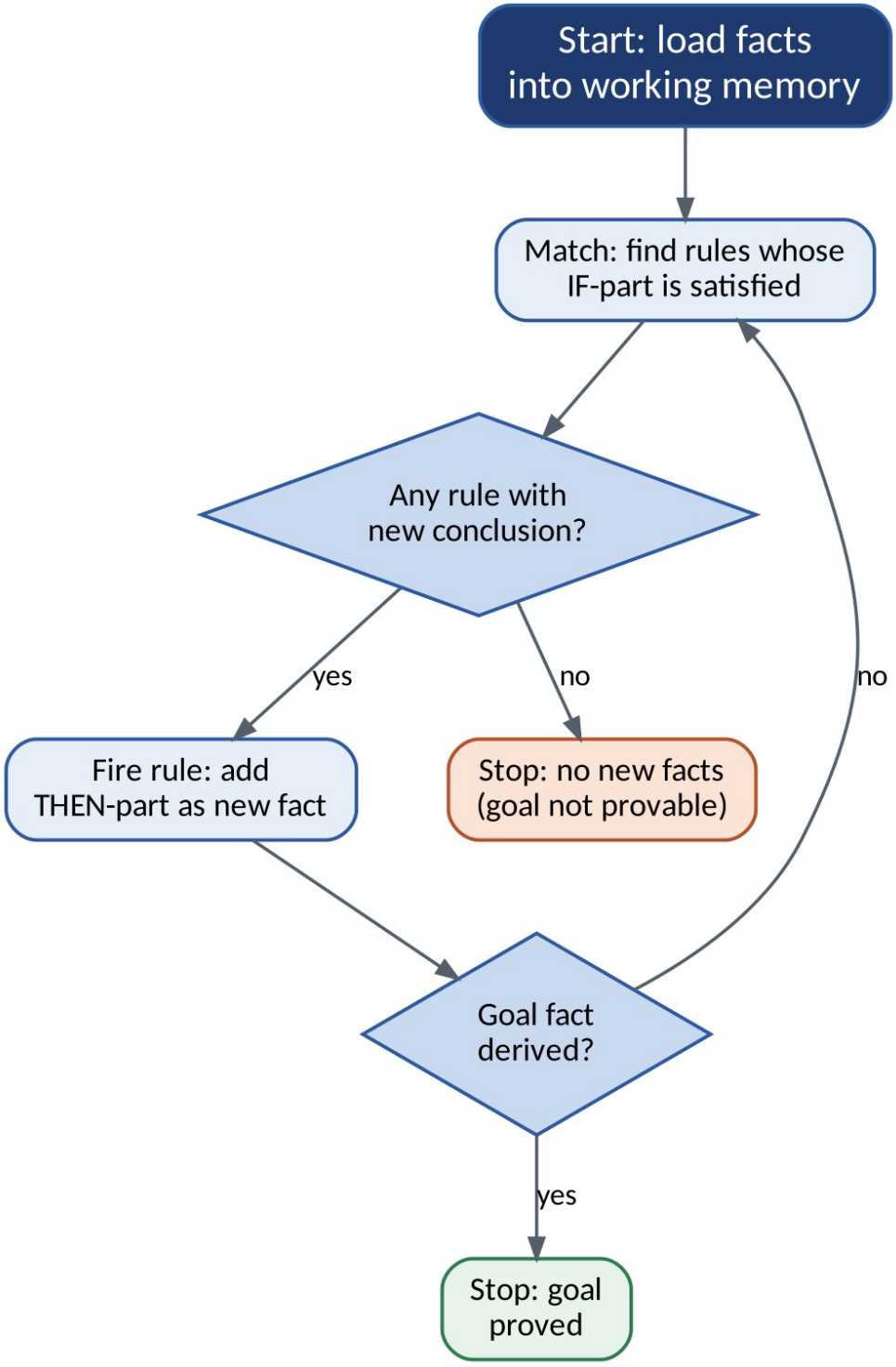


Figure 3.12: Flowchart of forward chaining

Forward Chaining Algorithm (Propositional, Definite Clauses)

```
function PL-FC-ENTAILS?(KB, q) returns true or false
   count  ← a table; count[c] = number of premises of rule c
   inferred ← a table; inferred[s] = false for every symbol s
   agenda ← a queue of symbols, initially the known facts in KB

   while agenda is not empty do
      p ← POP(agenda)
```

```
      if p = q then return true
      if inferred[p] = false then
         inferred[p] ← true
         for each rule c in KB where p is in c.PREMISE do
            decrement count[c]
            if count[c] = 0 then add c.CONCLUSION to agenda
   return false
```

The count of each rule records how many of its premises are still unknown; when it reaches 0 the rule fires. Each rule is processed at most once per premise, so the algorithm runs in time **linear** in the size of the KB.

Worked Example 1: A Propositional Rule Base

This is a standard textbook example.

```
R1: P → Q
R2: L ∧ M → P
R3: B ∧ L → M
R4: A ∧ P → L
R5: A ∧ B → L
Facts: A, B
Goal:  Q
```

Table 3.32: Forward chaining trace on the propositional rule base

Cycle	Known facts	Rule that fires	New fact
0	A, B	—	—
1	A, B	R5 (A ∧ B → L)	L
2	A, B, L	R3 (B ∧ L → M)	M
3	A, B, L, M	R2 (L ∧ M → P)	P
4	A, B, L, M, P	R1 (P → Q)	Q (goal)

R4 (A ∧ P → L) also becomes satisfied in cycle 4 but adds nothing new because L is already known. Q is derived, so **KB ⊨ Q**.

Worked Example 2: Forward Chaining in FOL — West is a Criminal

We use the same knowledge base as the resolution proof in the previous chapter, now written as definite clauses:

```
Rule 1: American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) → Criminal(x)
Rule 2: Missile(x) ∧ Owns(Nono, x) → Sells(West, x, Nono)
Rule 3: Missile(x) → Weapon(x)
Rule 4: Enemy(x, America) → Hostile(x)

Facts:  American(West), Missile(M1), Owns(Nono, M1), Enemy(Nono, America)
Goal:   Criminal(West)
```

Iteration 1 — every rule is tried against the initial facts.

- Rule 1: needs Weapon, Sells and Hostile facts — none known yet; cannot fire.
- Rule 2: Missile(M1) and Owns(Nono, M1) match with θ = {x/M1}; add **Sells(West, M1, Nono)**.
- Rule 3: Missile(M1) matches with θ = {x/M1}; add **Weapon(M1)**.
- Rule 4: Enemy(Nono, America) matches with θ = {x/Nono}; add **Hostile(Nono)**.

Iteration 2 — Rule 1 now matches American(West), Weapon(M1), Sells(West, M1, Nono), Hostile(Nono) with θ = {x/West, y/M1, z/Nono}; add **Criminal(West)**. The goal is reached.

Table 3.33: Forward chaining trace for "West is a criminal"

Iteration	Rule	Facts matched	Unifier θ	New fact
1	Rule 2	Missile(M1), Owns(Nono,M1)	{x/M1}	Sells(West, M1, Nono)
1	Rule 3	Missile(M1)	{x/M1}	Weapon(M1)
1	Rule 4	Enemy(Nono, America)	{x/Nono}	Hostile(Nono)
2	Rule 1	American(West), Weapon(M1), Sells(...), Hostile(Nono)	{x/West, y/M1, z/Nono}	Criminal(West)

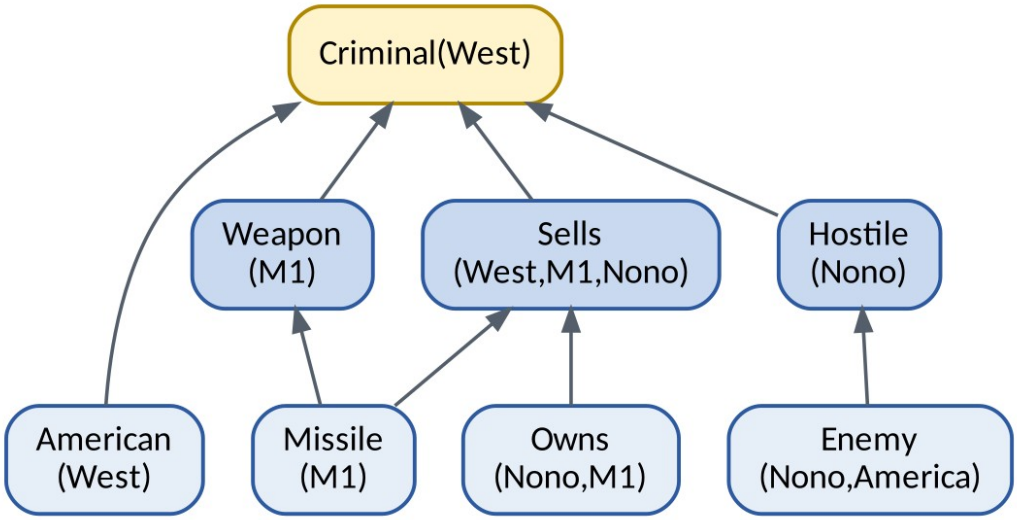


Figure 3.13: Forward chaining proof tree — initial facts at the bottom, goal at the top

The proof tree is read **bottom-up**: the four initial facts are at the bottom level, the facts inferred in iteration 1 are in the middle, and the goal inferred in iteration 2 is at the top.

Worked Example 3: Animal Identification

A small expert system identifies an animal from observations.

```
R1: has_hair                                  → mammal
R2: gives_milk                                → mammal
R3: has_feathers                              → bird
R4: mammal ∧ eats_meat                        → carnivore
R5: carnivore ∧ tawny_colour ∧ dark_spots      → cheetah
R6: carnivore ∧ tawny_colour ∧ black_stripes   → tiger

Facts: has_hair, eats_meat, tawny_colour, black_stripes
```

Worked Example — Forward chaining identifies a tiger: Cycle 1: R1 matches has_hair, so mammal is added (R2 and R3 do not match). Cycle 2: R4 matches mammal and eats_meat, so carnivore is added. Cycle 3: R5 needs dark_spots, which is not known, so it fails; R6 matches carnivore, tawny_colour and black_stripes, so tiger is added. No further rule adds a new fact, so the system stops and reports "The animal is a tiger". The same rule base is coded in CLIPS in the next chapter, where the real run output confirms this order.

Conflict Resolution

When several rules are ready to fire in the same cycle, the set of such rules is called the **conflict set**. A **conflict resolution strategy** decides which one fires first:

Table 3.34: Conflict resolution strategies

Strategy	Rule chosen
Rule order (textual order)	The rule written first in the rule base
Specificity	The rule with the most (most specific) conditions
Recency	The rule that uses the most recently added facts
Refraction	A rule does not fire twice on the same facts
Priority / salience	The rule with the highest assigned priority

Properties of Forward Chaining

- **Sound** — uses only modus ponens.
- **Complete** for definite-clause knowledge bases: every entailed atomic fact is eventually derived.
- **Data-driven** — useful when new facts arrive continuously (monitoring, alarms, sensors).
- May derive many facts that are **irrelevant** to the goal.

Backward Chaining

Idea

Backward chaining (backward reasoning, goal-driven inference, top-down reasoning) starts from the **goal**. It looks for rules whose **conclusion** matches the goal; the **premises** of such a rule become new **sub-goals**. Each sub-goal is proved the same way, until every sub-goal is matched by a known fact. If a rule leads to a dead end, the system **backtracks** and tries another rule.

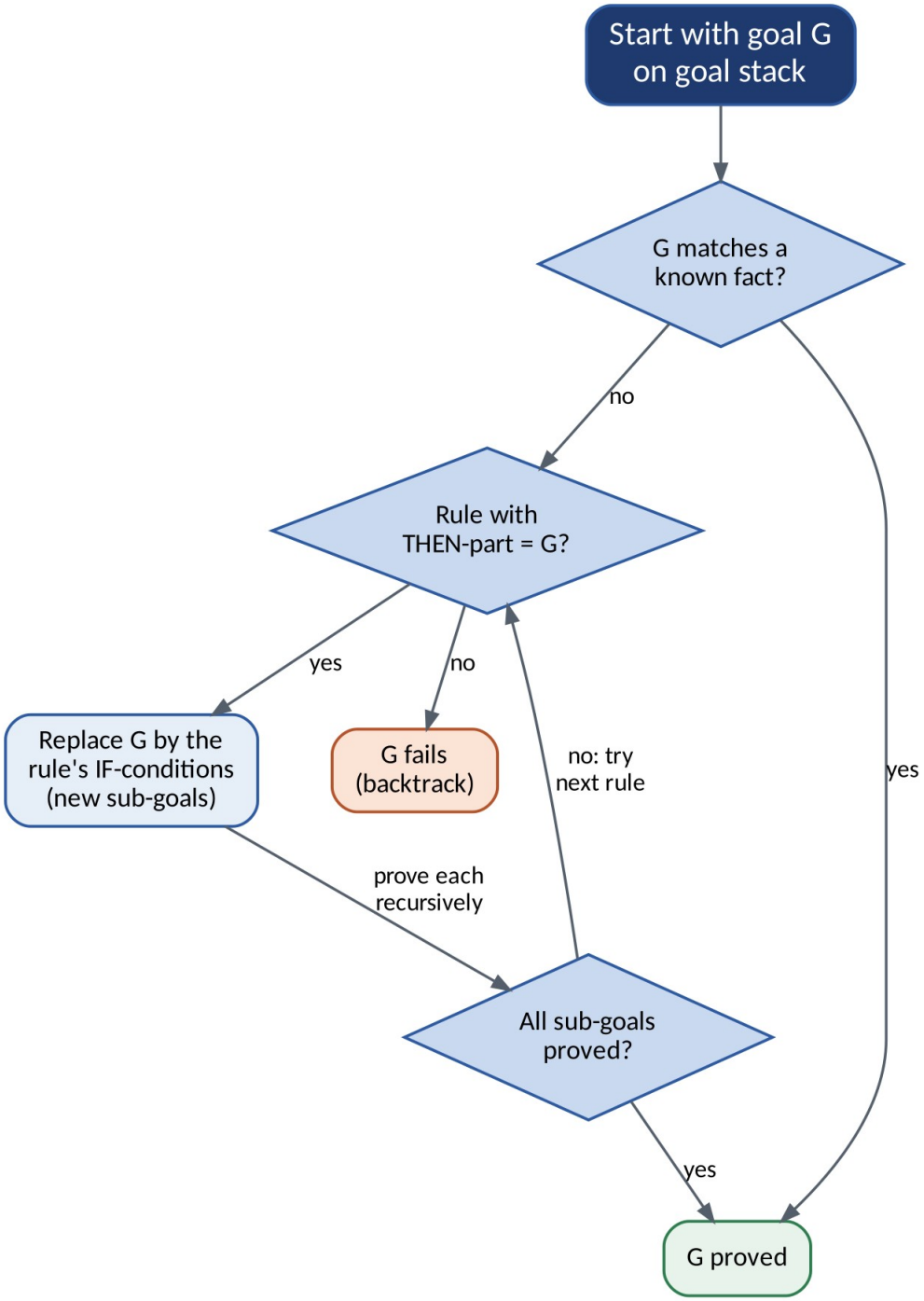


Figure 3.14: Flowchart of backward chaining

Backward Chaining Algorithm

```
function BC-PROVE(goal, θ) returns a set of substitutions
   // try facts first
   for each fact f in KB do
      θ' ← UNIFY(f, SUBST(θ, goal), θ)
      if θ' ≠ failure then yield θ'
   // then try rules whose conclusion unifies with the goal
   for each rule (p1 ∧ ... ∧ pn → q) in KB do
      (standardize the rule's variables apart)
      θ' ← UNIFY(q, SUBST(θ, goal), θ)
      if θ' ≠ failure then
```

```
        for each θ'' in BC-PROVE-ALL([p1, ..., pn], θ') do
           yield θ''

function BC-PROVE-ALL(goals, θ) returns a set of substitutions
   if goals is empty then yield θ
   else
      for each θ' in BC-PROVE(FIRST(goals), θ) do
         for each θ'' in BC-PROVE-ALL(REST(goals), θ') do
            yield θ''
```

It is a **depth-first** search: sub-goals are proved left to right, and substitutions found for earlier sub-goals are passed on to later ones. This is exactly how **Prolog** executes programs.

Worked Example 1: The Propositional Rule Base

Using R1–R5 and facts A, B from forward chaining Example 1, prove Q:

```
Goal Q
 └ R1: P → Q            sub-goal P
    └ R2: L ∧ M → P      sub-goals L, M
       ├ L: R4 (A ∧ P → L) needs P again → loop! skip, try R5
       │  R5: A ∧ B → L   A ✓ fact, B ✓ fact      → L proved
       └ M: R3: B ∧ L → M  B ✓ fact, L ✓ (already proved) → M proved
    P proved
Q proved
```

The trace shows an important detail: rule R4 would make the goal L depend on P, which is already being proved, causing an **infinite loop**. A practical backward chainer keeps a list of goals on the current path and skips any goal that repeats.

Worked Example 2: Backward Chaining in FOL — West is a Criminal

Table 3.35: Backward chaining trace for "West is a criminal"

Step	Current goal	Matched with	Unifier	New sub-goals / result
1	Criminal(West)	Rule 1	{x/West}	American(West), Weapon(y), Sells(West,y,z), Hostile(z)
2	American(West)	Fact	{ }	proved
3	Weapon(y)	Rule 3	{x/y}	Missile(y)
4	Missile(y)	Fact Missile(M1)	{y/M1}	proved, y = M1
5	Sells(West, M1, z)	Rule 2	{x/M1, z/Nono}	Missile(M1), Owns(Nono, M1)
6	Missile(M1)	Fact	{ }	proved
7	Owns(Nono, M1)	Fact	{ }	proved
8	Hostile(Nono)	Rule 4	{x/Nono}	Enemy(Nono, America)
9	Enemy(Nono, America)	Fact	{ }	proved

All sub-goals succeed, with final substitution {y/M1, z/Nono}. Hence **Criminal(West)** is proved.

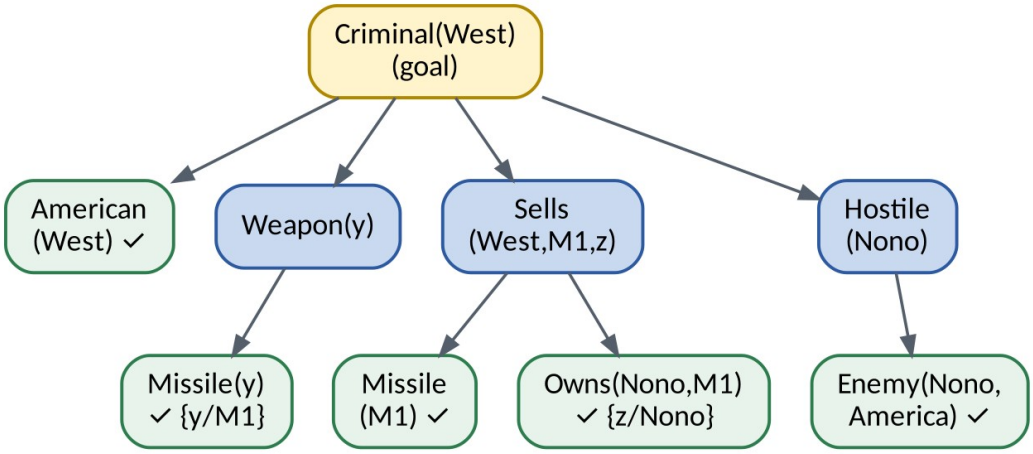


Figure 3.15: Backward chaining proof tree — goal at the root, facts at the leaves

The proof tree is built **top-down** and explored **depth-first, left to right**. Notice how the binding y = M1 found while proving Weapon(y) is carried into the next sub-goal, which becomes Sells(West, M1, z).

Worked Example 3: Is the Animal a Tiger?

Worked Example — Backward chaining on the animal rule base: Goal: tiger. R6 concludes tiger, so the sub-goals are carnivore, tawny_colour, black_stripes. Sub-goal carnivore: R4 concludes it, giving sub-goals mammal and eats_meat. Sub-goal mammal: R1 concludes it, giving sub-goal has_hair, which is a fact, so mammal is proved. eats_meat is a fact, so carnivore is proved. tawny_colour and black_stripes are facts. All sub-goals hold, so "tiger" is proved. Rules R2, R3 and R5 were never examined — backward chaining looks only at rules relevant to the goal.

In an interactive expert system, when a sub-goal is neither a fact nor the conclusion of any rule, the system **asks the user** ("Does the animal eat meat?"). MYCIN, the medical diagnosis system, worked this way.

Properties of Backward Chaining

- **Sound**, and complete for definite clauses if repeated states are checked.
- **Goal-driven** — examines only rules relevant to the goal, so it is usually faster for answering a specific question.
- Implemented as **depth-first search**, so it uses little memory but can fall into **infinite loops** and may repeat work (solved by **memoization**).
- The basis of **logic programming** (Prolog).

Forward vs Backward Chaining

Table 3.36: Forward chaining vs backward chaining

Basis	Forward chaining	Backward chaining
Starting point	Known facts	Goal (hypothesis)
Direction	Facts → conclusion (bottom-up)	Goal → facts (top-down)
Also called	Data-driven reasoning	Goal-driven reasoning
Search style	Breadth-first, all rules tried each cycle	Depth-first, one goal at a time

Basis	Forward chaining	Backward chaining
Rule matching	Facts matched to IF-part (LHS)	Goal matched to THEN-part (RHS)
Relevance	May derive many irrelevant facts	Only goal-relevant rules used
Efficiency	Slower for one specific query	Faster for one specific query
Best when	Many possible conclusions, few facts arrive	Few goals, many facts
Typical use	Monitoring, planning, design, alarms	Diagnosis, debugging, query answering
Example systems	CLIPS, OPS5, XCON (R1)	Prolog, MYCIN
Risk	Combinatorial growth of facts	Infinite loops on recursive rules

Memory Trick: "Forward = Facts First; Backward = Begin with the goal." Also: a Doctor uses backward chaining (Diagnosis — suspects a disease, then checks for its symptoms), while a burglar alarm uses forward chaining (a sensor fires, so the alarm rings).

Exam Tip: For a 10-mark question on chaining, use one rule base for both methods: write the rules and facts, give the forward-chaining iteration table, the backward-chaining sub-goal table, draw both proof trees, and finish with at least eight rows of the comparison table.

Applications of Chaining

- **Expert systems** — medical diagnosis (MYCIN used backward chaining), computer configuration (XCON used forward chaining).
- **Business rules engines** — loan approval, insurance claim checking, fraud alerts.
- **Monitoring and control** — process plants, network intrusion detection: new sensor data trigger rules (forward).
- **Logic programming and deductive databases** — Prolog queries (backward), Datalog (forward).
- **Games and chatbots** — rule-based behaviour of non-player characters.

Summary

- Both chaining methods work on definite clauses using (generalized) modus ponens; both are sound and complete for such KBs.
- Forward chaining: data-driven; fires every rule whose premises are known until the goal or a fixed point appears.
- Backward chaining: goal-driven; replaces a goal with the premises of a rule that concludes it, depth-first with backtracking.
- "West is a criminal" was proved in two iterations by forward chaining and in nine goal steps by backward chaining.
- Forward suits monitoring and many conclusions; backward suits diagnosis and answering specific questions.

AI Languages and Tools: Lisp, Prolog and CLIPS

Why Special Languages for AI?

Early AI programs manipulated **symbols** (words, concepts, rules) rather than numbers, and needed to build and change complex structures such as trees, lists and rule sets while the program was running. Ordinary languages of the 1950s–70s (Fortran, COBOL) were designed for numeric or business data and were poor at this. AI languages therefore offered:

- **Symbolic processing** — symbols and lists as basic data types.
- **Dynamic memory** — automatic storage allocation and **garbage collection**.
- **Recursion** — natural for trees, search and logic.
- **Interactive development** — a read–evaluate–print loop (REPL) for experimenting.
- **Built-in reasoning** — pattern matching, unification, backtracking or rule engines.

Table 3.37: Paradigms of AI languages and tools

Paradigm	Language / tool	Core idea
Functional	Lisp (and Scheme, Clojure)	Programs are functions applied to lists
Logic	Prolog	Programs are facts and rules; execution is proof
Rule-based (production system)	CLIPS (also OPS5, Jess, Drools)	IF–THEN rules fired by a forward-chaining engine
General-purpose with AI libraries	Python	Libraries for ML, NLP and search

Lisp

History and Features

Lisp (LISt Processing) was designed by **John McCarthy** in **1958** at MIT and described in his 1960 paper "Recursive Functions of Symbolic Expressions and Their Computation by Machine". It is the second-oldest high-level programming language still in wide use (after Fortran), and for decades it was the main language of AI research in the USA.

Important dialects:

- **Common Lisp** — the standard industrial dialect (ANSI standard, 1994). Implementations: SBCL, CLISP.
- **Scheme** — a small, clean dialect (1975), popular in teaching.
- **Emacs Lisp** — the extension language of the Emacs editor.
- **Clojure** — a modern Lisp running on the Java Virtual Machine.

Key features:

- **Code and data have the same form** (lists). A program can build and run other programs — useful for AI systems that learn or rewrite rules.
- **Prefix notation**: the operator comes first: (+ 2 3).
- **Recursion** as the main control structure.
- **Automatic garbage collection** — Lisp introduced it.
- **Dynamic typing** and an interactive **REPL**.

Key Point: In Lisp, everything is either an atom or a list, and a program is itself a list. This "code is data" property is why Lisp was the favourite language for symbolic AI.

S-Expressions: Atoms and Lists

All Lisp code and data are written as **S-expressions** (symbolic expressions). An S-expression is either:

- an **atom** — a number (42, 3.14), a symbol (apple, x, factorial), or a string ("hello"); or
- a **list** — zero or more S-expressions inside parentheses: (a b c), (1 (2 3) 4), ().

The empty list () is the same as the special symbol **NIL**, which also means "false". The symbol **T** means "true".

When Lisp evaluates a list, it treats the **first element as a function** and the rest as **arguments**:

```
(+ 2 3 4)          → 9
(* 2 (+ 3 4))      → 14
```

To stop evaluation and use a list as data, put a **quote** before it: '(a b c) means "the list a b c", not "call function a".

Basic List Functions: CAR, CDR, CONS

Table 3.38: Basic Lisp list functions with outputs

Function	Meaning	Example	Result
CAR (FIRST)	First element of a list	(car '(a b c))	A
CDR (REST)	List without its first element	(cdr '(a b c))	(B C)
CONS	Add an element at the front	(cons 'x '(a b))	(X A B)
CONS	Build a one-element list	(cons 'a nil)	(A)
CADR	CAR of the CDR (second element)	(cadr '(a b c))	B
LIST	Make a list of its arguments	(list 1 2 3)	(1 2 3)
APPEND	Join lists	(append '(1 2) '(3 4))	(1 2 3 4)
REVERSE	Reverse a list	(reverse '(red green blue))	(BLUE GREEN RED)
ATOM	Is it an atom?	(atom 'a) / (atom '(a b))	T / NIL
NULL	Is it the empty list?	(null nil)	T
EQUAL	Structural equality	(equal '(1 2) '(1 2))	T
MEMBER	Tail starting at the element	(member 'green '(red green blue))	(GREEN BLUE)

(Common Lisp prints symbols in capitals, which is why 'a prints as A.) The names CAR and CDR come from the IBM 704 machine on which Lisp was first implemented ("contents of the address/decrement part of register").

Memory Trick: "CAR is the head of the train, CDR is the rest of the carriages, CONS couples a new engine at the front." And (cons (car L) (cdr L)) always gives back L.

Defining Functions: DEFUN

Functions are defined with **defun**:

```
(defun function-name (parameters)
   body)
```

Example — square of a number, and applying it to every element with **mapcar**:

```
(defun square (x)
  (* x x))

(square 7)                          → 49
(mapcar #'square '(1 2 3 4))        → (1 4 9 16)
```

Conditionals: IF and COND

```
(if test then-part else-part)

(defun grade (m)
  (cond ((>= m 75) 'distinction)
        ((>= m 60) 'first)
        ((>= m 40) 'pass)
        (t 'fail)))

(grade 82)   → DISTINCTION
(grade 45)   → PASS
(grade 30)   → FAIL
```

cond tests each clause in turn and returns the value of the first one whose test is true; the final t clause acts as "else".

Recursion Example 1: Factorial

```
(defun factorial (n)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))

(factorial 5)    → 120
(factorial 10)   → 3628800
```

Trace of (factorial 5): 5 × (factorial 4) = 5 × 4 × (factorial 3) = … = 5 × 4 × 3 × 2 × 1 = 120.

Recursion Example 2: Length of a List

```
(defun my-length (lst)
  (if (null lst)
      0
      (+ 1 (my-length (cdr lst)))))

(my-length '(a b c d))   → 4
(my-length nil)          → 0
```

Worked Example — Tracing (my-length '(a b c)): The list is not empty, so the result is 1 + (my-length '(b c)). That is 1 + (my-length '(c)), which is 1 + (my-length nil). The base case (null lst) returns 0. Unwinding: 0 + 1 = 1, then 2, then 3. The answer is 3. Every recursive list function in Lisp follows this pattern: test for the empty list (base case), process the CAR, recurse on the CDR.

More Recursive Functions

```
(defun sum-list (lst)
  (cond ((null lst) 0)
        (t (+ (car lst) (sum-list (cdr lst))))))
(sum-list '(10 20 30))       → 60
```

```
(defun my-reverse (lst)
  (if (null lst)
      nil
      (append (my-reverse (cdr lst)) (list (car lst)))))
(my-reverse '(1 2 3 4))      → (4 3 2 1)

(defun fib (n)
  (if (< n 2)
      n
      (+ (fib (- n 1)) (fib (- n 2)))))
(fib 10)                     → 55
```

Variables and a Tiny Knowledge Base

```
(defparameter *kb* '((dog mammal) (sparrow bird)))

(assoc 'dog *kb*)               → (DOG MAMMAL)
(cadr (assoc 'sparrow *kb*))    → BIRD
```

defparameter (or **defvar**) defines a global variable; **setq** assigns a value to an existing variable; **let** creates local variables:

```
(let ((a 3) (b 4))
  (sqrt (+ (* a a) (* b b))))   → 5.0
```

Applications of Lisp

- Early AI programs: SHRDLU (natural language), MACSYMA (computer algebra).
- Expert-system shells and AI research tools.
- Extension language of AutoCAD (AutoLISP) and Emacs.
- Modern use through Clojure in web and data applications.

Prolog

History and Features

Prolog (PROgramming in LOGic) was created in **1972** by **Alain Colmerauer** and **Philippe Roussel** at the University of Aix-Marseille, France, based on **Robert Kowalski's** procedural interpretation of Horn clauses. It became popular in Europe and Japan (Japan's Fifth Generation Computer project of the 1980s was built around logic programming). An ISO standard was published in 1995. A widely used free implementation is **SWI-Prolog**.

Prolog is a **declarative** language: the programmer states **what** is true (facts and rules) and asks questions (queries); the Prolog engine works out **how** to answer, using:

- **Unification** — pattern matching of terms (the same idea as FOL unification).
- **Backward chaining** — goals are proved from the head of rules to their bodies.
- **Depth-first search with backtracking** — alternatives are tried in the order they are written.

Facts, Rules and Queries

Table 3.39: Facts, rules and queries in Prolog

Element	Syntax	Meaning	Logic form
Fact	parent(tom, bob).	Tom is a parent of Bob	Parent(Tom, Bob)

Element	Syntax	Meaning	Logic form
Rule	father(X, Y) :- parent(X, Y), male(X).	X is father of Y if X is parent of Y and X is male	∀x∀y Parent(x,y) ∧ Male(x) → Father(x,y)
Query	?- father(tom, bob).	Is Tom the father of Bob?	Goal to prove

Syntax rules:

- Every clause ends with a **full stop**.
- **:-** is read "if"; a **comma** means "and"; a **semicolon** means "or".
- **Constants (atoms)** start with a lowercase letter: tom, bob.
- **Variables** start with an uppercase letter or underscore: X, Who, _ (anonymous).
- A rule's left side is the **head**, the right side the **body**.

Family-Tree Program

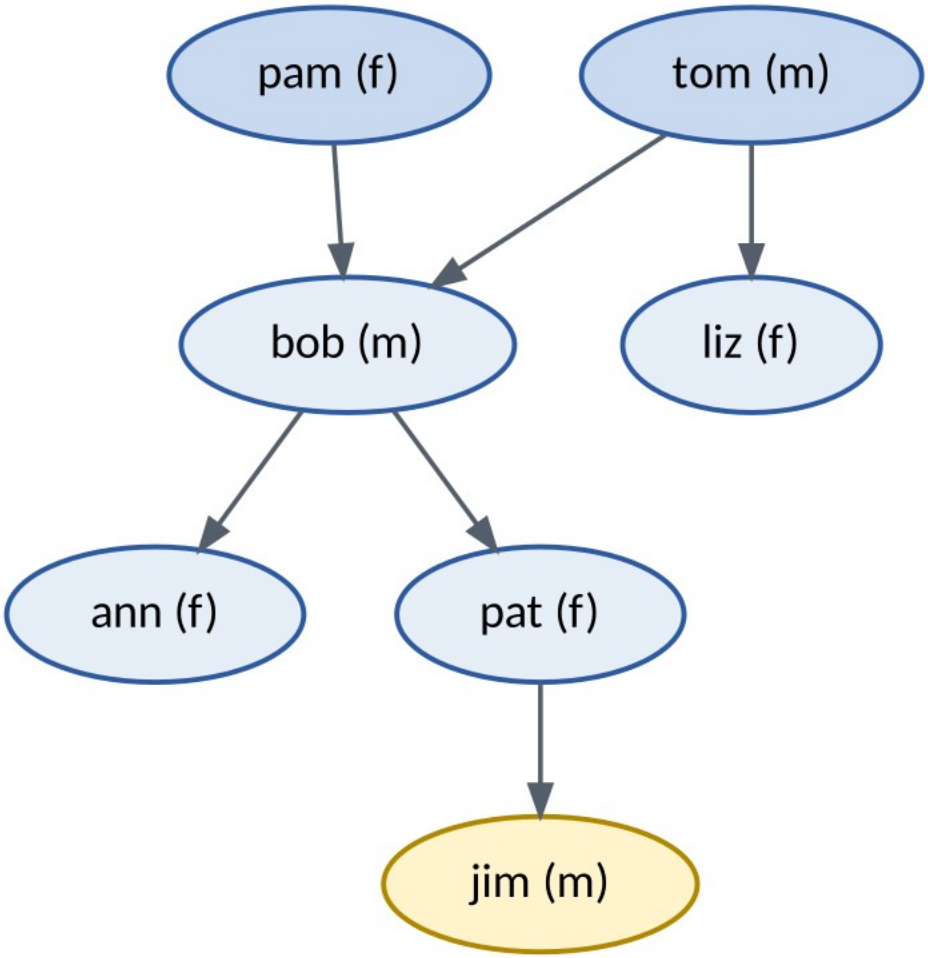


Figure 3.16: Family tree used in the Prolog program

```
% ----- facts -----
parent(tom, bob).
parent(pam, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
```

```
parent(pat, jim).

male(tom).
male(bob).
male(jim).
female(pam).
female(liz).
female(ann).
female(pat).

% ----- rules -----
father(X, Y)      :- parent(X, Y), male(X).
mother(X, Y)      :- parent(X, Y), female(X).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
sibling(X, Y)     :- parent(Z, X), parent(Z, Y), X \= Y.
sister(X, Y)      :- sibling(X, Y), female(X).

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

(The operator `\=` means "does not unify with", so a person is not counted as his or her own sibling.)

Queries and Answers

The answers below are those produced by SWI-Prolog. Typing ; after an answer asks for the next one; false means no more answers.

Table 3.40: Prolog queries on the family-tree program

Query	Answer(s)	Meaning
?- parent(tom, bob).	true	Tom is Bob's parent
?- parent(X, liz).	X = tom	Who is Liz's parent?
?- parent(liz, X).	false	Liz has no children
?- father(X, bob).	X = tom	Bob's father
?- mother(X, Y).	X = pam, Y = bob ; X = pat, Y = jim	All mother–child pairs
?- grandparent(tom, X).	X = ann ; X = pat	Tom's grandchildren
?- grandparent(X, jim).	X = bob	Jim's grandparent
?- sibling(ann, X).	X = pat	Ann's sibling
?- sibling(bob, X).	X = liz	Bob's sibling (via Tom)
?- sister(X, ann).	X = pat	Ann's sister
?- ancestor(tom, jim).	true	Tom is an ancestor of Jim
?- ancestor(X, jim).	X = pat ; X = tom ; X = pam ; X = bob	All ancestors of Jim

How Prolog Answers: Unification and Backtracking

Worked Example — Answering ?- grandparent(tom, X): Prolog unifies the goal with the head grandparent(X1, Z1), giving X1 = tom, Z1 = X. The body becomes parent(tom, Y), parent(Y, X). The first fact matching parent(tom, Y) is parent(tom, bob), so Y = bob; then parent(bob, X) matches parent(bob, ann), giving the first answer X = ann. When the user types ;, Prolog backtracks to the most recent choice and finds parent(bob, pat), giving X = pat. Backtracking further, parent(tom, Y) next gives Y = liz, but parent(liz, X) has no match, so Prolog answers false — no more solutions.

Backtracking means that when a goal fails (or the user asks for more answers), Prolog returns to the most recent **choice point** — a goal that had other matching clauses — undoes the variable bindings made after it, and tries the next alternative. This is depth-first search over the proof tree, i.e. **backward chaining**.

Recursion: the Ancestor Rule

```
ancestor(X, Y) :- parent(X, Y).                  % base case
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).   % recursive case
```

Trace of ?- ancestor(tom, jim):

```
ancestor(tom, jim)
  clause 1: parent(tom, jim)?                    no
  clause 2: parent(tom, Z), ancestor(Z, jim)
     Z = bob: ancestor(bob, jim)
        clause 1: parent(bob, jim)?              no
        clause 2: parent(bob, Z2), ancestor(Z2, jim)
           Z2 = ann: ancestor(ann, jim)          fails (ann has no children)
           Z2 = pat: ancestor(pat, jim)
              clause 1: parent(pat, jim)?        yes
true.
```

The base case must come first and the recursive call should come last in the body; writing ancestor(X, Y) :- ancestor(Z, Y), parent(X, Z). would make Prolog recurse forever (**left recursion**).

More Recursive Programs: Factorial and List Length

```
factorial(0, 1).
factorial(N, F) :- N > 0, N1 is N - 1, factorial(N1, F1), F is N * F1.

len([], 0).
len([_|T], N) :- len(T, N1), N is N1 + 1.

?- factorial(5, F).        F = 120
?- len([a,b,c,d], N).      N = 4
```

Notes:

- **is** evaluates arithmetic: N1 is N - 1. Plain = only unifies, it does not calculate.
- **[H|T]** splits a list into head H and tail T: [H|T] = [a,b,c] gives H = a, T = [b,c].

Unification in Prolog

The operator **=** asks Prolog to unify two terms:

Table 3.41: Unification in Prolog

Query	Result
?- f(X, b) = f(a, Y).	X = a, Y = b
?- p(X, X) = p(a, b).	false
?- [H|T] = [a, b, c].	H = a, T = [b, c]
?- X = point(1, Y), Y = 2.	X = point(1, 2), Y = 2
?- tom = bob.	false

(For speed, standard Prolog **omits the occurs check**; ISO Prolog provides unify_with_occurs_check/2 when it is needed.)

The Cut (!)

The **cut**, written **!**, is a built-in goal that always succeeds but **prevents backtracking** past the point where it appears. It commits Prolog to the choices made so far in that clause.

```
max_of(X, Y, X) :- X >= Y, !.
max_of(X, Y, Y) :- X < Y.

?- max_of(8, 2, M).     M = 8
?- max_of(4, 9, M).     M = 9
```

When X >= Y succeeds, the cut stops Prolog from trying the second clause. Cuts improve efficiency and can express "if–then–else", but careless use can change the logical meaning of a program, so they should be used sparingly.

> **Exam Tip:** A typical 10-mark Prolog question asks you to write a family database and rules for father, mother, grandparent, sibling and ancestor, then give the answers to some queries. Learn the program above by heart and practise tracing one query with backtracking.

Applications of Prolog

- Expert systems and rule-based decision support.
- Natural language processing (Prolog's Definite Clause Grammars).
- Theorem proving and symbolic reasoning.
- Database querying (Datalog is a subset of Prolog).
- Parts of IBM Watson used Prolog for question analysis.

CLIPS

History and Features

CLIPS (C Language Integrated Production System) is an **expert-system shell** developed at **NASA's Johnson Space Center** beginning in **1985**. It was written in C so that it would be portable, fast and easy to embed in other programs. Its syntax resembles Lisp (everything is in parentheses), and its design was inspired by the earlier OPS5 and ART systems. CLIPS is now in the public domain and is maintained as open-source software; the Java-based **Jess** is a well-known descendant.

CLIPS supports three programming styles:

- **Rule-based** — defrule (the main style).
- **Procedural** — deffunction, generic functions.
- **Object-oriented** — COOL (CLIPS Object-Oriented Language).

CLIPS uses **forward chaining**, and it matches rules against facts efficiently with the **Rete algorithm** (Charles Forgy). Rete remembers partial matches in a network, so on each cycle only the changes to working memory are processed instead of re-matching every rule against every fact.

CLIPS Architecture

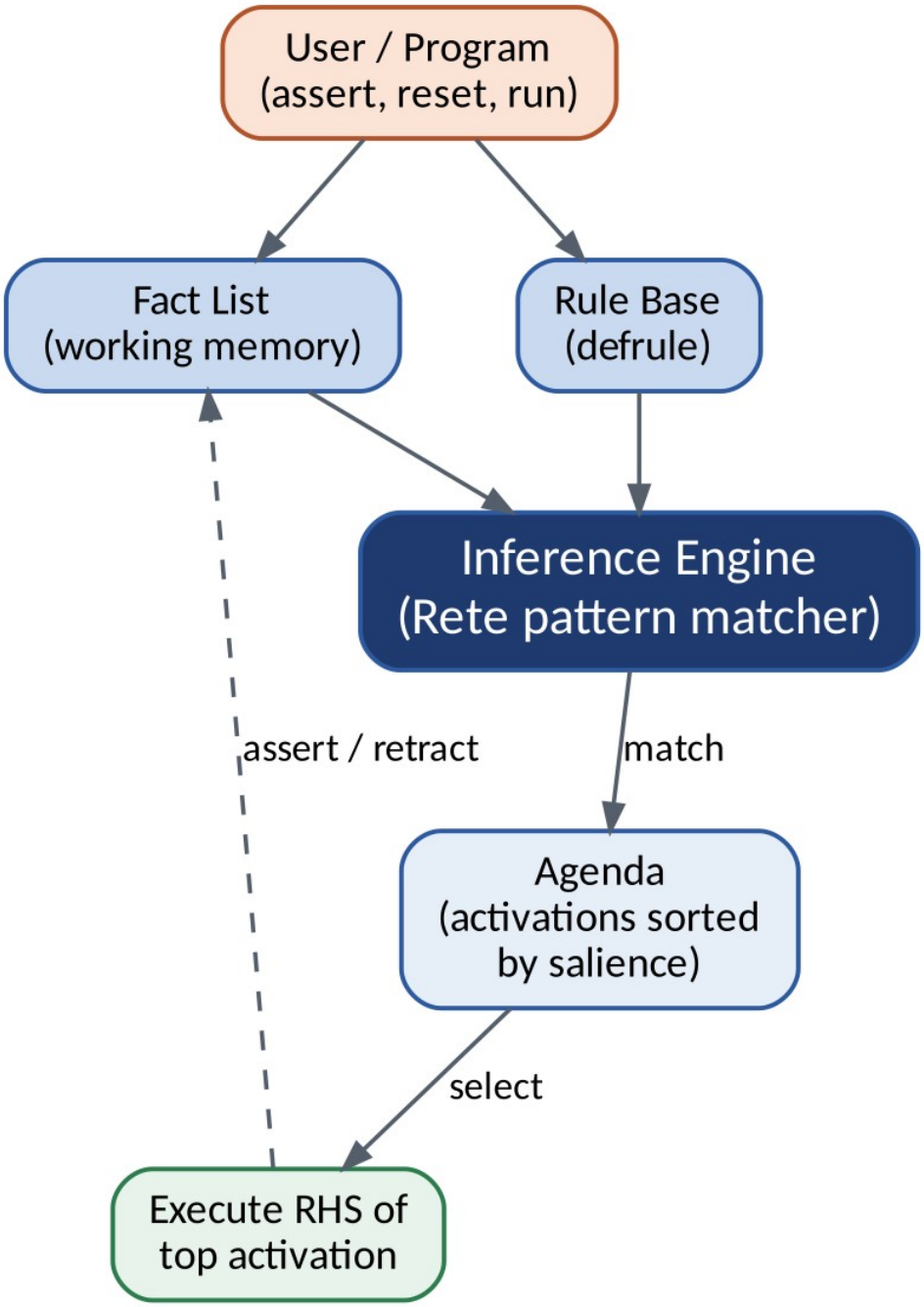


Figure 3.17: CLIPS architecture — fact list, rule base, agenda and inference engine

Table 3.42: Components of a CLIPS system

Component	Role
Fact list (working memory)	Holds the current facts, each with an index f-1, f-2, ...
Rule base (knowledge base)	Holds all defrule constructs
Inference engine	Matches facts to rule LHS patterns (Rete), selects and executes rules

Component	Role
Agenda	Ordered list of activations (rules whose LHS is satisfied, with the facts that matched)
Activation	One rule + one set of matching facts, ready to fire

Recognise–act cycle in CLIPS:

1. **Match** — the Rete network finds all rules whose LHS patterns are satisfied; these activations are placed on the **agenda**.
2. **Conflict resolution** — activations are ordered by **salience** (priority, default 0); among equal salience, the default **depth** strategy puts newer activations first.
3. **Act** — the top activation fires: its RHS actions run (they may assert, retract or modify facts).
4. Repeat until the agenda is empty or a (halt) is executed.

Facts and Templates

Ordered facts are simple lists:

```
(assert (has hair))
(assert (colour tawny))
```

Template (non-ordered) facts have named **slots**, defined with **deftemplate**:

```
(deftemplate person
   (slot name)
   (slot age (type INTEGER)))

(assert (person (name Ravi) (age 20)))
```

deffacts defines a group of initial facts that are asserted every time (reset) is executed.

Rules: defrule, LHS and RHS

```
(defrule rule-name
   "optional comment"
   <pattern-1>            ; LHS: conditions (patterns)
   <pattern-2>
   =>
   <action-1>             ; RHS: actions
   <action-2>)
```

- **LHS** (left-hand side) — patterns to be matched against facts. `?x` is a single-field variable, `?` a single-field wildcard and `$?` a multi-field wildcard; `&:` adds a test on a variable.
- **=>** separates LHS from RHS.
- **RHS** (right-hand side) — actions: (assert ...), (retract ...), (modify ...), (printout t ... crlf).

Example 1: Adult or Minor

```
(deftemplate person
   (slot name)
   (slot age (type INTEGER)))

(deffacts people
   (person (name Ravi) (age 20))
   (person (name Meena) (age 15)))

(defrule adult
   (person (name ?n) (age ?a&:(>= ?a 18)))
   =>
```

```
   (printout t ?n " is an adult." crlf))

(defrule minor
   (person (name ?n) (age ?a&:(< ?a 18)))
   =>
   (printout t ?n " is a minor." crlf))
```

Run (CLIPS 6.30):

```
CLIPS> (reset)
CLIPS> (facts)
f-0     (initial-fact)
f-1     (person (name Ravi) (age 20))
f-2     (person (name Meena) (age 15))
For a total of 3 facts.
CLIPS> (agenda)
0      minor: f-2
0      adult: f-1
For a total of 2 activations.
CLIPS> (run)
Meena is a minor.
Ravi is an adult.
```

Both activations have salience 0; under the default depth strategy the newer one (for fact f-2) is on top, so Meena is printed first. (CLIPS 6.4 and later no longer create the (initial-fact), so numbering starts at f-1 there.)

Example 2: Animal Identification by Forward Chaining

This is the rule base from the chaining chapter, written in CLIPS.

```
(deffacts animal-facts
   (has hair)
   (eats meat)
   (colour tawny)
   (pattern black-stripes))

(defrule r1-mammal
   (has hair)
   =>
   (assert (is mammal))
   (printout t "R1 fired: it is a mammal" crlf))

(defrule r4-carnivore
   (is mammal)
   (eats meat)
   =>
   (assert (is carnivore))
   (printout t "R4 fired: it is a carnivore" crlf))

(defrule r6-tiger
   (is carnivore)
   (colour tawny)
   (pattern black-stripes)
   =>
   (assert (animal tiger))
   (printout t "R6 fired: the animal is a TIGER" crlf))
```

Run output:

```
CLIPS> (reset)
CLIPS> (agenda)
0      r1-mammal: f-1
For a total of 1 activation.
CLIPS> (run)
```

```
R1 fired: it is a mammal
R4 fired: it is a carnivore
R6 fired: the animal is a TIGER
CLIPS> (facts)
f-0     (initial-fact)
f-1     (has hair)
f-2     (eats meat)
f-3     (colour tawny)
f-4     (pattern black-stripes)
f-5     (is mammal)
f-6     (is carnivore)
f-7     (animal tiger)
For a total of 8 facts.
```

Worked Example — Watching the agenda change: After (reset) only r1-mammal is on the agenda, because only its pattern (has hair) is satisfied. When it fires, it asserts (is mammal) as f-5; Rete immediately notices that r4-carnivore is now fully matched by f-5 and f-2 and puts it on the agenda. Firing r4 asserts f-6 (is carnivore), which activates r6-tiger. This step-by-step growth of the fact list from f-5 to f-7 is exactly forward chaining.

Example 3: Salience and the test Element

```
(deftemplate student
   (slot name)
   (slot marks (type INTEGER)))

(defrule greet
   (declare (salience 10))
   =>
   (printout t "Result processing started" crlf))

(defrule check-result
   (student (name ?n) (marks ?m))
   (test (>= ?m 40))
   =>
   (printout t ?n " has passed with " ?m " marks." crlf))

(defrule fail-result
   (student (name ?n) (marks ?m&:(< ?m 40)))
   =>
   (printout t ?n " has failed." crlf))

CLIPS> (reset)
CLIPS> (assert (student (name Asha) (marks 72)))
CLIPS> (assert (student (name Vikram) (marks 35)))
CLIPS> (run)
Result processing started
Vikram has failed.
Asha has passed with 72 marks.
```

The rule greet has no LHS patterns, so it is activated by (reset); its salience 10 puts it ahead of the other rules, which have the default salience 0.

Common CLIPS Commands

Table 3.43: Common CLIPS commands

Command	Purpose
(load "file.clp")	Load constructs from a file
(reset)	Clear the fact list and assert all deffacts
(run)	Start the inference engine

Command	Purpose
(facts)	Show the fact list
(agenda)	Show activations waiting to fire
(assert (fact))	Add a fact
(retract n)	Remove fact number n
(watch facts) / (watch rules)	Trace changes while running
(clear)	Remove all constructs and facts
(exit)	Leave CLIPS

Comparison: Lisp vs Prolog vs CLIPS

Table 3.44: Lisp vs Prolog vs CLIPS

Basis	Lisp	Prolog	CLIPS
Full form	LISt Processing	PROgramming in LOGic	C Language Integrated Production System
Year and origin	1958, John McCarthy, MIT	1972, Colmerauer and Roussel, Marseille	1985, NASA Johnson Space Center
Paradigm	Functional, symbolic	Logic (declarative)	Rule-based production system
Basic unit	Function, list (S-expression)	Fact, rule (Horn clause)	Fact, defrule, deftemplate
Reasoning built in	None (programmer writes it)	Backward chaining with unification	Forward chaining with Rete
Control	Recursion, explicit calls	Depth-first search, backtracking	Recognise–act cycle, agenda, salience
Variables	Symbols bound by let, defun	Uppercase names, bound by unification	?x pattern variables
Best for	Symbol manipulation, prototyping AI tools	Querying knowledge, NLP, theorem proving	Expert systems, monitoring, configuration
Implementation examples	SBCL, CLISP, Clojure	SWI-Prolog, GNU Prolog	CLIPS, Jess (Java)

> **Memory Trick:** "Lisp Lists, Prolog Proves, CLIPS Clips rules forward." Years in order: Lisp '58, Prolog '72, CLIPS '85 — each about a decade or more apart.

Other Tools: Python for AI

Today **Python** is the most widely used language for AI and machine learning. It is not a special "AI language" like Lisp or Prolog, but it has a simple syntax and a very large set of libraries:

Table 3.45: Python libraries for AI

Area	Python libraries / tools
Numerical computing	NumPy, SciPy
Data handling	pandas
Machine learning	scikit-learn

Area	Python libraries / tools
Deep learning	TensorFlow, PyTorch, Keras
Natural language processing	NLTK, spaCy
Logic and rules	SymPy (logic module), pyDatalog, PySWIP (Prolog bridge)

A forward-chaining rule engine is only a few lines of Python:

```
rules = [({"has_hair"}, "mammal"),
         ({"mammal", "eats_meat"}, "carnivore"),
         ({"carnivore", "tawny", "black_stripes"}, "tiger")]
facts = {"has_hair", "eats_meat", "tawny", "black_stripes"}

changed = True
while changed:
    changed = False
    for premises, conclusion in rules:
        if premises <= facts and conclusion not in facts:
            facts.add(conclusion)
            print("Derived:", conclusion)
            changed = True
```

Output:

```
Derived: mammal
Derived: carnivore
Derived: tiger
```

Other tools worth knowing by name: **Java** (Jess, Drools rule engines), **R** (statistics), **Julia** (high-performance numerical work), and knowledge-representation tools such as **Protégé** (ontology editor using OWL).

> **Key Point:** Lisp, Prolog and CLIPS each build one AI idea into the language — symbol processing, logical inference and production rules respectively. Python instead provides these ideas through libraries, which is why modern AI work is mostly done in Python.

Summary

- Lisp (1958, McCarthy): S-expressions, atoms and lists; CAR, CDR, CONS; defun; recursion; code is data.
- Prolog (1972, Colmerauer): facts, rules, queries; unification; backward chaining with depth-first backtracking; recursion (ancestor); cut (!) prevents backtracking.
- CLIPS (NASA, 1985): deftemplate, deffacts, defrule with LHS => RHS; agenda ordered by salience; forward chaining with the Rete algorithm.
- Lisp is functional, Prolog is logic-based and goal-driven, CLIPS is rule-based and data-driven.
- Python dominates modern AI because of its libraries.

Unit 3 Cheat Sheet

Knowledge Representation in One Page

- **KR** = writing knowledge in a formal form (syntax + semantics) so a program can reason with it.
- **Types of knowledge:** declarative (what), procedural (how), meta (about knowledge), heuristic (rules of thumb), structural (relations).
- **Good KR system:** representational adequacy, inferential adequacy, inferential efficiency, acquisitional efficiency.
- **KB agent:** KB + inference engine; TELL adds sentences, ASK queries; levels = knowledge, logical, implementation.

Table 3.46: KR approaches at a glance

Approach	Unit of knowledge	Key feature
Logic	Sentence	Precise semantics, sound inference
Semantic net	Node + labelled arc	is-a / instance-of inheritance
Frame	Slots + fillers	Defaults, overriding, demons
Production rule	IF–THEN	Modular; used by chaining and CLIPS
Script	Scenes, roles, props	Stereotyped event sequence

Inference and Reasoning

- **KB ⊨ α**: α true in every model of KB. **KB ⊢ α**: α derivable.
- **Sound**: derives only entailed sentences. **Complete**: derives all entailed sentences.
- **Deduction** general → specific (certain); **induction** specific → general (probable); **abduction** effect → best cause (plausible).
- **Monotonic**: conclusions never withdrawn. **Non-monotonic**: may be withdrawn (defaults).

Propositional Logic

Table 3.47: All connectives in one truth table

P	Q	¬P	P ∧ Q	P ∨ Q	P → Q	P ↔ Q
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

- Precedence: ( ) > ¬ > ∧ > ∨ > → > ↔ .
- Tautology (always T), contradiction (always F), contingency (mixed).
- P → Q ≡ ¬P ∨ Q ≡ ¬Q → ¬P. De Morgan: ¬(P ∧ Q) ≡ ¬P ∨ ¬Q; ¬(P ∨ Q) ≡ ¬P ∧ ¬Q.
- KB ⊨ α iff KB ∧ ¬α is unsatisfiable.

Table 3.48: Inference rules in one line each

Rule	Form
Modus ponens	$P \to Q, P \vdash Q$
Modus tollens	$P \to Q, \neg Q \vdash \neg P$
Hypothetical syllogism	$P \to Q, Q \to R \vdash P \to R$
Disjunctive syllogism	$P \vee Q, \neg P \vdash Q$
And-elimination	$P \wedge Q \vdash P$
Resolution	$P \vee Q, \neg P \vee R \vdash Q \vee R$

- **CNF steps:** remove $\leftrightarrow$ $\to$ remove $\to$ $\to$ push $\neg$ inwards $\to$ distribute $\vee$ over $\wedge$.
- **Resolution refutation:** CNF(KB) + $\neg$goal $\to$ resolve until $\square$.
- **Horn clause:** at most one positive literal; **definite clause:** exactly one.

First-Order Logic

- Elements: constants, variables, predicates, functions, connectives, quantifiers, equality.
- $\forall$ with $\to$, $\exists$ with $\wedge$. $\neg\forall x\ P \equiv \exists x\ \neg P$; $\neg\exists x\ P \equiv \forall x\ \neg P$.
- $\forall x\ \exists y$ Loves(x, y) $\neq \exists y\ \forall x$ Loves(x, y).
- **UI:** $\forall x\ \alpha \Rightarrow \alpha\{x/\text{any ground term}\}$. **EI:** $\exists x\ \alpha \Rightarrow \alpha\{x/\text{new constant}\}$, once.
- **Unification:** find θ with SUBST(θ, p) = SUBST(θ, q); **MGU** = most general one; fails on different constants, different predicates, or occurs check.
- **GMP:** p1', ..., pn', (p1 $\wedge$ ... $\wedge$ pn $\to$ q) $\vdash$ SUBST(θ, q).
- **FOL CNF:** remove $\to$, push $\neg$, standardize apart, Skolemize, drop $\forall$, distribute.

Chaining

Table 3.49: Forward vs backward chaining summary

Feature	Forward	Backward
Starts from	Facts	Goal
Driven by	Data	Goal
Search	Breadth-like, all rules	Depth-first, backtracking
Matches	Facts to IF-part	Goal to THEN-part
Used in	CLIPS, XCON, monitoring	Prolog, MYCIN, diagnosis

- West KB, forward: iteration 1 $\to$ Weapon(M1), Sells(West,M1,Nono), Hostile(Nono); iteration 2 $\to$ Criminal(West).

AI Languages

Table 3.50: Lisp, Prolog and CLIPS at a glance

Item	Lisp	Prolog	CLIPS
Year / creator	1958, McCarthy	1972, Colmerauer	1985, NASA JSC
Paradigm	Functional	Logic	Rule-based
Key syntax	(defun f (x) ...)	head :- body.	(defrule r LHS => RHS)

Item	Lisp	Prolog	CLIPS
Inference	Programmer-written	Backward + unification	Forward + Rete

- Lisp: (car '(a b c)) → A; (cdr '(a b c)) → (B C); (cons 'x '(a b)) → (X A B); factorial 5 → 120.
- Prolog: facts end with "."; :- means if; variables start uppercase; ! (cut) stops backtracking.
- CLIPS: (reset) loads deffacts; (run) fires agenda; salience sets priority; Rete matches efficiently.

Memory Tricks

- **D-P-M-H-S** ("Do People Make Happy Systems"): declarative, procedural, meta, heuristic, structural knowledge.
- **"RI-IA"** for KR properties: Representational adequacy, Inferential adequacy, Inferential efficiency, Acquisitional efficiency.
- **"Only T → F is False"** for implication.
- **"All-Arrow, Exists-And"**: ∀ with →, ∃ with ∧.
- **"Sound = Safe, Complete = Catches all."**
- **CNF = "I-I-N-D"**: Iff out, Implies out, Negation in, Distribute.
- **FOL CNF = "I-N-S-S-D-D"**: Implications, Negation, Standardize, Skolemize, Drop ∀, Distribute.
- **"Forward = Facts First; Backward = Begin with the goal."**
- **"Lisp Lists, Prolog Proves, CLIPS Clips rules forward"** — '58, '72, '85.

Important Questions

Two-Mark Questions (with Model Answers)

Q1. What is knowledge representation? Knowledge representation is the field of AI concerned with expressing knowledge about the world in a formal, machine-usable form (with syntax and semantics) so that a program can reason with it and solve problems.

Q2. Distinguish declarative and procedural knowledge. Declarative knowledge is "knowing what" — facts such as "Jaipur is in Rajasthan". Procedural knowledge is "knowing how" — steps or methods such as an algorithm for sorting.

Q3. What is a semantic network? A semantic network is a graph in which nodes represent objects or concepts and labelled arcs represent relations among them, such as is-a, instance-of and has. It supports inheritance of properties.

Q4. What is a frame? A frame is a record-like structure (Minsky, 1974) describing a stereotyped object, made of slots and fillers. Slots can hold default values, which can be inherited and overridden, and procedural attachments such as IF-NEEDED.

Q5. What are TELL and ASK? They are the two operations of a knowledge-based agent. TELL adds a new sentence to the knowledge base; ASK queries the knowledge base, and the answer must follow from what was TELLed.

Q6. Define soundness and completeness of an inference procedure. A procedure is sound if every sentence it derives is entailed by the KB. It is complete if it can derive every sentence that is entailed by the KB.

Q7. What is a tautology? Give an example. A tautology (valid sentence) is a sentence that is true in every model, i.e. every row of its truth table is T. Example: $P \vee \neg P$.

Q8. State modus ponens and modus tollens. Modus ponens: from $P \rightarrow Q$ and P infer Q. Modus tollens: from $P \rightarrow Q$ and $\neg Q$ infer $\neg P$.

Q9. What is CNF? Conjunctive Normal Form is a conjunction (AND) of clauses, where each clause is a disjunction (OR) of literals, e.g. $(\neg A \vee B) \wedge (C \vee D)$. Resolution requires sentences in CNF.

Q10. Write "All students are intelligent" and "Some students are lazy" in FOL. $\forall x\ \text{Student}(x) \rightarrow \text{Intelligent}(x)$. $\exists x\ \text{Student}(x) \wedge \text{Lazy}(x)$.

Q11. What is unification? What is an MGU? Unification finds a substitution that makes two expressions identical. The Most General Unifier is the unifier that makes the fewest commitments; e.g. the MGU of Knows(John, x) and Knows(y, Bill) is {y/John, x/Bill}.

Q12. Differentiate forward and backward chaining in one line each. Forward chaining is data-driven: it starts from known facts and fires rules to reach the goal. Backward chaining is goal-driven: it starts from the goal and works back to supporting facts.

Q13. What do CAR, CDR and CONS do in Lisp? CAR returns the first element of a list, CDR returns the rest of the list, and CONS adds an element to the front: (car '(a b c)) → A, (cdr '(a b c)) → (B C), (cons 'x '(a b)) → (X A B).

Q14. What is the cut (!) in Prolog? The cut is a built-in goal that always succeeds but prevents Prolog from backtracking past it, committing to the choices made in that clause. It is used for efficiency and if–then–else behaviour.

Q15. What is CLIPS? Who developed it? CLIPS (C Language Integrated Production System) is a forward-chaining, rule-based expert-system shell developed at NASA's Johnson Space Center (from 1985). It uses the Rete algorithm to match facts to rules.

Five-Mark Questions (with Answer Outlines)

Q1. Explain the types of knowledge with examples. Outline: definition of knowledge; five types — declarative (facts), procedural (how-to), meta-knowledge (about knowledge), heuristic (rules of thumb), structural (relations); one example each; Table 3.4 style comparison.

Q2. What are the properties of a good knowledge representation system? Outline: representational adequacy, inferential adequacy, inferential efficiency, acquisitional efficiency — one line each with an example; add naturalness, modularity, clear semantics.

Q3. Explain semantic networks with a diagram. State advantages and disadvantages. Outline: nodes and labelled arcs; is-a and instance-of links; draw the animal net (Figure 3.4); show inheritance by answering "Does Tom have fur?"; advantages (natural, inheritance, compact); disadvantages (no standard link meaning, weak on negation and quantifiers).

Q4. Differentiate deduction, induction and abduction. Outline: definition and direction of each; one example each (Socrates, swans, wet grass); certainty and soundness; uses (theorem proving, machine learning, diagnosis); comparison table (Table 3.10).

Q5. Explain the rules of inference in propositional logic. Outline: define inference rule; state modus ponens, modus tollens, hypothetical syllogism, disjunctive syllogism, and-elimination, resolution, each with an English example; note all are sound.

Q6. Explain universal and existential instantiation with examples. Outline: UI — substitute any ground term, can be repeated (King/Greedy/Evil example); EI — substitute a new Skolem constant, applied once (Crown example); comparison table; mention propositionalization.

Q7. Compare forward and backward chaining. Outline: definitions; direction; search strategy; efficiency; best use cases; example systems (CLIPS, XCON vs Prolog, MYCIN); at least eight rows of Table 3.36; one tiny example.

Q8. Write short notes on CLIPS: facts, rules and the agenda. Outline: origin at NASA; ordered vs template facts (deftemplate); defrule syntax LHS => RHS; agenda, activation and salience; Rete-based forward chaining; one small rule with output; architecture diagram (Figure 3.17).

Ten-Mark Questions (with Structured Answer Plans)

Q1 (Problem). Convert to CNF and prove by resolution

"If Ravi studies, he passes. If Ravi does not play, he studies. Ravi did not pass." Prove that Ravi played.

Solution. Symbols: S = Ravi studies, P = Ravi passes, L = Ravi plays.

```
KB:     S → P,    ¬L → S,    ¬P
Goal:   L

CNF conversion:
  S → P    ⇒  ¬S ∨ P                    (C1)
  ¬L → S   ⇒  ¬¬L ∨ S  ⇒  L ∨ S         (C2)
  ¬P                                     (C3)
Negated goal:  ¬L                        (C4)

Resolution:
  C5 = resolve(C4, C2) on L :  S
```

```
C6 = resolve(C5, C1) on S :  P
C7 = resolve(C6, C3) on P :  □   (empty clause)
```

The empty clause shows KB ∧ ¬L is unsatisfiable, so **Ravi played** is proved.

Answer plan for full marks: (1) define CNF and the four conversion steps, (2) symbolize, (3) convert each sentence with steps shown, (4) negate goal, (5) resolution table and tree, (6) conclusion with one line on soundness and refutation completeness.

Q2 (Problem). FOL resolution: the Marcus example

Given: (1) Marcus was a man. (2) Marcus was a Pompeian. (3) All Pompeians were Romans. (4) Caesar was a ruler. (5) All Romans were either loyal to Caesar or hated him. (6) Everyone is loyal to someone. (7) People only try to assassinate rulers they are not loyal to. (8) Marcus tried to assassinate Caesar. (9) All men are people. Prove by resolution: **Marcus hated Caesar.**

Step 1 — FOL:

```
1. Man(Marcus)
2. Pompeian(Marcus)
3. ∀x Pompeian(x) → Roman(x)
4. Ruler(Caesar)
5. ∀x Roman(x) → Loyalto(x, Caesar) ∨ Hate(x, Caesar)
6. ∀x ∃y Loyalto(x, y)
7. ∀x ∀y Person(x) ∧ Ruler(y) ∧ Tryassassinate(x, y) → ¬Loyalto(x, y)
8. Tryassassinate(Marcus, Caesar)
9. ∀x Man(x) → Person(x)
```

Step 2 — Clauses (variables standardized apart; f is a Skolem function from sentence 6):

```
C1: Man(Marcus)
C2: Pompeian(Marcus)
C3: ¬Pompeian(x1) ∨ Roman(x1)
C4: Ruler(Caesar)
C5: ¬Roman(x2) ∨ Loyalto(x2, Caesar) ∨ Hate(x2, Caesar)
C6: Loyalto(x3, f(x3))
C7: ¬Person(x4) ∨ ¬Ruler(y1) ∨ ¬Tryassassinate(x4, y1) ∨ ¬Loyalto(x4, y1)
C8: Tryassassinate(Marcus, Caesar)
C9: ¬Man(x5) ∨ Person(x5)
Negated goal  C10: ¬Hate(Marcus, Caesar)
```

Step 3 — Resolution:

Table 3.51: Resolution proof that Marcus hated Caesar

Step	Resolve	Unifier	Resolvent
1	C10, C5	{x2/Marcus}	¬Roman(Marcus) ∨ Loyalto(Marcus, Caesar)
2	Step 1, C3	{x1/Marcus}	¬Pompeian(Marcus) ∨ Loyalto(Marcus, Caesar)
3	Step 2, C2	{ }	Loyalto(Marcus, Caesar)
4	Step 3, C7	{x4/Marcus, y1/Caesar}	¬Person(Marcus) ∨ ¬Ruler(Caesar) ∨ ¬Tryassassinate(Marcus, Caesar)
5	Step 4, C4	{ }	¬Person(Marcus) ∨ ¬Tryassassinate(Marcus, Caesar)

Step	Resolve	Unifier	Resolvent
6	Step 5, C8	{ }	¬Person(Marcus)
7	Step 6, C9	{x5/Marcus}	¬Man(Marcus)
8	Step 7, C1	{ }	□

The empty clause is derived, so **Hate(Marcus, Caesar)** is proved. Clause C6 was not needed.

Answer plan: (1) FOL with predicate definitions, (2) CNF steps briefly (Skolemization of sentence 6), (3) clause list, (4) negated goal, (5) proof table or tree with unifiers, (6) conclusion.

Q3 (Problem). Forward and backward chaining on one rule base

Rule base: R1: A ∧ B → C; R2: C ∧ D → E; R3: B ∧ E → F; R4: F → G. Facts: A, B, D. Show that G can be derived using (a) forward chaining and (b) backward chaining.

(a) Forward chaining:

Table 3.52: Forward chaining derivation of G

Cycle	Facts known	Rule fired	New fact
1	A, B, D	R1	C
2	A, B, D, C	R2	E
3	A, B, D, C, E	R3	F
4	A, B, D, C, E, F	R4	G (goal)

(b) Backward chaining:

```
Goal G
 └ R4: needs F
    └ R3: needs B, E
       ├ B ✓ fact
       └ E: R2 needs C, D
            ├ C: R1 needs A, B → A ✓, B ✓  → C proved
            └ D ✓ fact                     → E proved
       F proved
 G proved
```

Answer plan: (1) define both strategies, (2) forward table, (3) backward goal tree, (4) draw both proof trees, (5) comparison table of at least six rows, (6) applications.

Q4. Explain Prolog with a family-tree program

Answer plan:

1. **Introduction** — Prolog, 1972, Colmerauer and Roussel; declarative logic programming; based on Horn clauses.
2. **Elements** — facts, rules, queries; syntax (full stop, :-, comma, variables in uppercase).
3. **Program** — the parent/male/female facts and the father, mother, grandparent, sibling and ancestor rules (as in Chapter "AI Languages and Tools").
4. **Queries with answers** — at least five from Table 3.40, e.g. ?- grandparent(tom, X). gives X = ann ; X = pat.
5. **Execution** — unification, backward chaining, depth-first search, backtracking with one traced query.
6. **Recursion** — ancestor rule with base and recursive case; warning about left recursion.
7. **Cut** — max_of example.

8. **Applications and comparison** with Lisp and CLIPS (Table 3.44).

MCQ Practice

1. Which type of knowledge is "knowing how" to do something? (a) Declarative (b) Procedural (c) Meta-knowledge (d) Structural

2. Knowledge about knowledge is called: (a) Heuristic knowledge (b) Structural knowledge (c) Meta-knowledge (d) Declarative knowledge

3. The property of a KR system that allows new knowledge to be derived is: (a) Inferential adequacy (b) Acquisitional efficiency (c) Representational adequacy (d) Naturalness

4. In a semantic network, properties are passed from a class to its members through: (a) has links (b) part-of links (c) is-a and instance-of links (d) owned-by links

5. Frames were introduced by: (a) John McCarthy (b) Marvin Minsky (c) Alan Turing (d) Alain Colmerauer

6. A script is best suited to represent: (a) Numerical data (b) A stereotyped sequence of events (c) A truth table (d) A single fact

7. The two operations of a knowledge-based agent are: (a) PUSH and POP (b) READ and WRITE (c) TELL and ASK (d) MATCH and FIRE

8. An inference procedure that derives only entailed sentences is: (a) Complete (b) Sound (c) Monotonic (d) Satisfiable

9. "The grass is wet, so it probably rained" is an example of: (a) Deduction (b) Induction (c) Abduction (d) Resolution

10. $P \rightarrow Q$ is false only when: (a) P is F and Q is T (b) P is T and Q is F (c) both are F (d) both are T

11. Which connective has the highest precedence? (a) $\wedge$ (b) $\vee$ (c) $\rightarrow$ (d) $\neg$

12. $P \vee \neg P$ is a: (a) Contradiction (b) Contingency (c) Tautology (d) Unsatisfiable sentence

13. $P \rightarrow Q$ is logically equivalent to: (a) $Q \rightarrow P$ (b) $\neg P \rightarrow \neg Q$ (c) $\neg Q \rightarrow \neg P$ (d) $P \wedge \neg Q$

14. From $P \vee Q$ and $\neg P$ we infer Q. This rule is: (a) Modus ponens (b) Disjunctive syllogism (c) Hypothetical syllogism (d) And-elimination

15. By De Morgan's law, $\neg(P \wedge Q)$ is equivalent to: (a) $\neg P \wedge \neg Q$ (b) $\neg P \vee \neg Q$ (c) $P \vee Q$ (d) $\neg P \vee Q$

16. The first step in converting a propositional sentence to CNF is: (a) Distribute $\vee$ over $\wedge$ (b) Move $\neg$ inwards (c) Eliminate $\leftrightarrow$ (d) Skolemize

17. Resolution refutation proves $KB \models \alpha$ by showing that: (a) $KB \wedge \alpha$ is valid (b) $KB \wedge \neg\alpha$ is unsatisfiable (c) α is satisfiable (d) $\neg\alpha$ is valid

18. A Horn clause has: (a) Exactly two positive literals (b) At most one positive literal (c) No negative literals (d) Only negative literals

19. Which is the correct FOL for "Some students are lazy"? (a) $\forall x$ Student(x) $\rightarrow$ Lazy(x) (b) $\exists x$ Student(x) $\rightarrow$ Lazy(x) (c) $\exists x$ Student(x) $\wedge$ Lazy(x) (d) $\forall x$ Student(x) $\wedge$ Lazy(x)

20. $\neg\forall x\, P(x)$ is equivalent to: (a) $\forall x \neg P(x)$ (b) $\exists x \neg P(x)$ (c) $\neg\exists x\, P(x)$ (d) $\exists x\, P(x)$

21. In Existential Instantiation the variable is replaced by: (a) Any existing constant (b) A new Skolem constant (c) A variable (d) Every ground term

22. The MGU of Knows(John, x) and Knows(y, Mother(y)) is: (a) {x/John, y/Mother(John)} (b) {y/John, x/Mother(John)} (c) {x/y} (d) Not unifiable

23. P(x) and P(f(x)) cannot be unified because of the: (a) Arity check (b) Occurs check (c) Predicate check (d) Constant check

24. Forward chaining is also called: (a) Goal-driven reasoning (b) Top-down reasoning (c) Data-driven reasoning (d) Abductive reasoning

25. Prolog answers queries using: (a) Forward chaining with Rete (b) Backward chaining with depth-first search (c) Breadth-first search (d) Truth tables

26. Lisp was designed by John McCarthy in: (a) 1950 (b) 1958 (c) 1972 (d) 1985

27. In Common Lisp, (cdr '(a b c)) returns: (a) A (b) (A B) (c) (B C) (d) C

28. In Prolog, ":-" is read as: (a) and (b) or (c) if (d) not

29. CLIPS was developed at: (a) MIT (b) University of Marseille (c) NASA Johnson Space Center (d) Stanford University

30. In CLIPS, the order of activations on the agenda is first decided by: (a) Rule name (b) Salience (c) Number of facts (d) File order only

Answer Key

Table 3.53: MCQ answer key

Q	Ans	Explanation
1	b	Procedural knowledge is knowing how (steps, methods).
2	c	Meta-knowledge is knowledge about knowledge.
3	a	Inferential adequacy means structures can be manipulated to derive new knowledge.
4	c	Inheritance flows along is-a and instance-of links.
5	b	Minsky proposed frames in 1974.
6	b	Scripts (Schank and Abelson) describe event sequences such as a restaurant visit.
7	c	TELL adds sentences; ASK queries the KB.
8	b	Sound = truth-preserving; derives only entailed sentences.
9	c	Reasoning from effect to likely cause is abduction.
10	b	The only false row of implication is T → F.
11	d	Negation binds tightest after parentheses.
12	c	P ∨ ¬P is true in every model.
13	c	The contrapositive is equivalent to the implication.

Q	Ans	Explanation
14	b	Disjunctive syllogism eliminates the false disjunct.
15	b	¬(P ∧ Q) ≡ ¬P ∨ ¬Q.
16	c	Biconditionals are removed first, then implications.
17	b	KB ⊨ α iff KB ∧ ¬α is unsatisfiable (derives □).
18	b	Horn clause: at most one positive literal.
19	c	∃ is used with ∧.
20	b	Quantifier De Morgan: ¬∀x P ≡ ∃x ¬P.
21	b	EI uses a brand-new constant, applied once.
22	b	y binds to John first, so x becomes Mother(John).
23	b	A variable cannot unify with a term containing itself.
24	c	Forward chaining starts from data (facts).
25	b	Prolog uses goal-driven, depth-first backward chaining with backtracking.
26	b	McCarthy designed Lisp at MIT in 1958.
27	c	CDR returns the list without its first element.
28	c	head :- body means head is true if body is true.
29	c	CLIPS was built at NASA's Johnson Space Center from 1985.
30	b	Higher salience activations are placed first; ties use the conflict strategy.

Glossary

Table 3.54: Glossary of Unit 3 terms

Term	Meaning
Abduction	Reasoning from an observed effect to its most likely cause; plausible, not guaranteed
Agenda	In CLIPS, the ordered list of rule activations waiting to fire
Atom (Lisp)	An indivisible S-expression: a number, symbol or string
Backtracking	Returning to the latest choice point to try an alternative when a goal fails
Backward chaining	Goal-driven inference that works from the goal back to known facts
CAR / CDR	Lisp functions returning the first element / the rest of a list
Clause	A disjunction of literals, e.g. ¬P ∨ Q ∨ R
CLIPS	C Language Integrated Production System; NASA's forward-chaining expert-system shell
CNF	Conjunctive Normal Form: an AND of clauses
Completeness	Ability of an inference procedure to derive every entailed sentence
CONS	Lisp function that adds an element to the front of a list
Contradiction	A sentence false in every model; unsatisfiable
Cut (!)	Prolog goal that succeeds once and blocks backtracking past it
Deduction	Reasoning from general premises to a certain specific conclusion
Definite clause	A clause with exactly one positive literal; an IF–THEN rule
Entailment	KB ⊨ α: α is true in every model in which KB is true
Existential instantiation	Replacing an ∃-quantified variable by a new Skolem constant
Forward chaining	Data-driven inference that fires rules from known facts towards the goal
Frame	Slot–filler structure representing a stereotyped object, with defaults and inheritance
Generalized modus ponens	Lifted modus ponens for FOL using a unifying substitution
Horn clause	A clause with at most one positive literal
Induction	Reasoning from specific cases to a general rule; probable
Inference engine	The component that derives new knowledge from the knowledge base
Knowledge base	A set of sentences in a representation language
Literal	An atomic sentence or its negation
MGU	Most General Unifier: the least restrictive substitution making two expressions identical

Term	Meaning
Modus ponens	From P → Q and P infer Q
Non-monotonic reasoning	Reasoning in which conclusions may be withdrawn when new facts arrive
Occurs check	Unification test forbidding a variable to bind to a term containing itself
Predicate	A symbol denoting a property or relation, returning true or false
Production rule	An IF condition THEN action rule
Resolution	Inference rule combining two clauses with complementary literals into a resolvent
Rete algorithm	Efficient pattern-matching algorithm used by CLIPS to match facts with rules
Salience	Priority of a CLIPS rule; higher salience fires first
S-expression	Lisp's universal syntax: an atom or a parenthesised list
Semantic network	Graph of concepts (nodes) connected by labelled relations (arcs)
Skolem function	A function replacing an ∃ variable that depends on enclosing ∀ variables
Soundness	Property of deriving only entailed (true) sentences
Tautology	A sentence true in every model; a valid sentence
Unification	Finding a substitution that makes two logical expressions identical